

SOLIDWORKS API (.NET VBA C#) — Examples (3/50): Add and Remove Items to File Save As and Add Menu and Menu Item Example CSharp

SOLIDWORKS API Documentation (converted from CHM)

Add and Remove Items to File Save As and Open Menus Example (VB.NET)

This example shows how to add items to the **File > Save As** and **File > Open** file type menus.

```

'-----
' Preconditions:
' 1. In Microsoft Visual Studio 2010, or later, create a SOLIDWORKS add-in
'    project using Visual Basic > SwVBAddin.
' 2. Name the project SwVBAddin1.
' 3. Copy and paste this code into SwAddin.vb of your VB.NET project.
' 4. Select Project > SwVBAddin1 Properties > Debug.
' 5. Select Start external program and type the path to sldworks.exe. Include
'    sldworks.exe in the path.
' 6. Press F5 to start debugging SOLIDWORKS Design with this add-in.
'-----
' Postconditions:
' 1. Open a part.
' 2. Click File > Save As.
' 3. Select XXX file (*.xxx) in Save as type.
' 4. Click Save.
' 5. Displays the path and file name to which to save the part.
' 6. Click OK to close the message box.
'-----

```

[SwAddin.vb](#):

```

Imports System
Imports System.Collections
Imports System.Reflection
Imports System.Runtime.InteropServices

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swpublished
Imports SOLIDWORKSTools
Imports SOLIDWORKSTools.File

```

```

<Guid("a3ddcdce-c8a8-47c2-8561-919a24285286")> _
<ComVisible(True)> _
<SwAddin( _
    Description:="SwVBAddin1 third-party pop-up menus", _
    Title:="SwVBAddin1", _
    LoadAtStartup:=True _
)> _
Public Class SwAddin
Implements SolidWorks.Interop.swpublished.SwAddin

```

```

#Region "Local Variables"
Dim WithEvents iSwApp As SldWorks
Dim iCmdMgr As ICommandManager
Dim addinID As Integer
Dim openDocs As Hashtable
Dim SwEventPtr As SldWorks
Dim ppage As UserPMPPage
Dim registerID As Integer
Dim resultCode As Boolean
Dim model As ModelDoc2
Dim frame As IFrame
Dim cmdGroup As ICommandGroup

' Public Properties
ReadOnly Property SwApp() As SldWorks
Get
    Return iSwApp
End Get
End Property

ReadOnly Property CmdMgr() As ICommandManager
Get
    Return iCmdMgr
End Get
End Property

ReadOnly Property OpenDocumentsTable() As Hashtable
Get
    Return openDocs
End Get
End Property
#End Region

```

```

#Region "SOLIDWORKS Registration"

```

```

<ComRegisterFunction()> Public Shared Sub RegisterFunction(ByVal t As Type)

' Get Custom Attribute: SwAddinAttribute
Dim attributes() As Object
Dim SwAttr As SwAddinAttribute = Nothing

```

```

attributes = System.Attribute.GetCustomAttributes(GetType(SwAddin), GetType(SwAddinAttribute))

If attributes.Length > 0 Then
    SwAttr = DirectCast(attributes(0), SwAddinAttribute)
End If

Dim hklm As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.LocalMachine
Dim hkcu As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.CurrentUser

Dim keyname As String = "SOFTWARE\SOLIDWORKS\Addins\{" + t.GUID.ToString() + "}"
Dim addinkey As Microsoft.Win32.RegistryKey = hklm.CreateSubKey(keyname)
addinkey.SetValue(Nothing, 0)
addinkey.SetValue("Description", SwAttr.Description)
addinkey.SetValue("Title", SwAttr.Title)

keyname = "Software\SOLIDWORKS\AddInsStartup\{" + t.GUID.ToString() + "}"
addinkey = hkcu.CreateSubKey(keyname)
addinkey.SetValue(Nothing, SwAttr.LoadAtStartup, Microsoft.Win32.RegistryValueKind.DWord)
End Sub

<ComUnregisterFunction()> Public Shared Sub UnregisterFunction(ByVal t As Type)
    Dim hklm As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.LocalMachine
    Dim hkcu As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.CurrentUser

    Dim keyname As String = "SOFTWARE\SOLIDWORKS\Addins\{" + t.GUID.ToString() + "}"
    hklm.DeleteSubKey(keyname)

    keyname = "Software\SOLIDWORKS\AddInsStartup\{" + t.GUID.ToString() + "}"
    hkcu.DeleteSubKey(keyname)
End Sub

#End Region

#Region "ISwAddin Implementation"

Function ConnectToSW(ByVal ThisSW As Object, ByVal Cookie As Integer) As Boolean Implements
SolidWorks.Interop.swpublished.SwAddin.ConnectToSW
    iSwApp = ThisSW
    addinID = Cookie

    ' Set up callbacks
    iSwApp.SetAddinCallbackInfo(0, Me, addinID)

    'Set up File > Save as type item and File > Open > File name type item
    bRet = iSwApp.AddFileSaveAsItem2(addinID, "XXX_FileSave", "XXX file (*.xxx)", "xxx", swDocumentTypes_e.swDocPART)
    bRet = iSwApp.AddFileOpenItem3(addinID, "XXX_FileOpen", "XXX file (*.xxx)", "xxx", "", "")

    ' Set up the Command Manager
    iCmdMgr = iSwApp.GetCommandManager(Cookie)
    AddCommandMgr()

    'Set up the Event Handlers
    SwEventPtr = iSwApp
    openDocs = New Hashtable
    AttachEventHandlers()

    'Set up Sample Property Manager
    AddPMP()

    ConnectToSW = True
End Function

Function DisconnectFromSW() As Boolean Implements SolidWorks.Interop.swpublished.SwAddin.DisconnectFromSW
    bRet = iSwApp.RemoveFileSaveAsItem2(addinID, "XXX_FileSave", "XXX file (*.xxx)", "xxx", swDocumentTypes_e.swDocPART)
    bRet = iSwApp.RemoveFileOpenItem2(addinID, "XXX_FileOpen", "XXX file (*.xxx)", "xxx")

    RemoveCommandMgr()
    RemovePMP()
    DetachEventHandlers()

    iSwApp = Nothing
    'The addin _must_ call GC.Collect() here in order to retrieve all managed code pointers
    GC.Collect()
    DisconnectFromSW = True
End Function

Public Sub XXX_FileSave(sFileName As String)
    ' S_OK = Saved successfully
    ' S_FALSE = Unsuccessful
    ' !(SUCCEEDED) = Insuccessful

    Dim fileOp As Integer = 1
    ParseFilename(sFileName, fileOp)
End Sub

Public Sub XXX_FileOpen(sFileName As String)
    ' S_OK = Loaded successfully
    ' S_FALSE = Unsuccessful
    ' !(SUCCEEDED) = Unsuccessful
    Dim fileOp As Integer = 0
    ParseFilename(sFileName, fileOp)
End Sub

Public Function ParseFilename(ByVal sFileName As String, ByRef fileOp As Integer) As String
    Dim tempParseFileName As String
    Dim strExtension As String
    Dim lPos As Integer

```

```

Dim lNumExtensionOccurrences As Integer
Dim lStart As Integer
Dim strSearchString As String
Dim lNumNonRealExtensions As Integer
Dim strExtensionWithPeriod As String

' Set return value
tempParseFilename = ""

' Set the extension for which to look
strExtension = "xxx"
strExtensionWithPeriod = "." & strExtension

' Strip the trailing 'w' or 'r' and any leading and trailing white space
sFileName = Trim$(Left$(sFileName, Len(sFileName) - 1))

' Strip extension from the back
sFileName = Trim$(Left$(sFileName, Len(sFileName) - Len(strExtension)))

' Change to lowercase to make search case-insensitive
strSearchString = LCase$(sFileName)

lNumExtensionOccurrences = 0

lStart = 1

Do

    lPos = InStr(lStart, strSearchString, LCase$(strExtensionWithPeriod))

    If (lPos > 0) Then

        lNumExtensionOccurrences = lNumExtensionOccurrences + 1

        ' Move start point of search
        lStart = lPos + Len(strExtensionWithPeriod)

    End If

Loop While (lPos > 0)

' There is 1 real extension and n*2 non-real extension
lNumNonRealExtensions = (lNumExtensionOccurrences \ 2)

' Start searching from the end to locate the real extension
' Skip the number of non-real extensions, before reaching the real extension

' Change to lowercase to make search case-insensitive
strSearchString = LCase$(sFileName)

lStart = -1

Do

    lPos = InStrRev(strSearchString, LCase$(strExtensionWithPeriod), lStart)

    If (lPos > 0) Then

        lNumNonRealExtensions = lNumNonRealExtensions - 1

        ' Move start point of search
        lStart = lPos - 1

    End If

Loop While ((lPos > 0) And (lNumNonRealExtensions >= 0))

'lStart points to the start of real extension
sFileName = Left$(sFileName, lStart) & strExtensionWithPeriod

If fileOp = 1 Then
    MsgBox("XXX_FileSave = " & sFileName)
Else
    MsgBox("XXX_FileOpen = " & sFileName)
End If

End Function

#End Region

#Region "UI Methods"

Public Sub AddCommandMgr()

    Dim iBmp As New BitmapHandler
    Dim thisAssembly As Assembly

    Dim cmdIndex0 As Integer, cmdIndex1 As Integer
    Dim Title As String = "VB.NET Add-in"
    Dim ToolTip As String = "VB.NET Add-in"

    Dim docTypes() As Integer = {swDocumentTypes_e.swDocASSEMBLY, _
        swDocumentTypes_e.swDocDRAWING, _
        swDocumentTypes_e.swDocPART}

    thisAssembly = System.Reflection.Assembly.GetAssembly(Me.GetType())

    cmdGroup = iCmdMgr.CreateCommandGroup(1, Title, ToolTip, "", -1)
    cmdGroup.LargeIconList = iBmp.CreateFileFromResourceBitmap("SwVBAddin1.ToolbarLarge.bmp", thisAssembly)
    cmdGroup.SmallIconList = iBmp.CreateFileFromResourceBitmap("SwVBAddin1.ToolbarSmall.bmp", thisAssembly)

```

```

cmdGroup.LargeMainIcon = iBmp.CreateFileFromResourceBitmap("SwVbAddin1.MainIconLarge.bmp", thisAssembly)
cmdGroup.SmallMainIcon = iBmp.CreateFileFromResourceBitmap("SwVbAddin1.MainIconSmall.bmp", thisAssembly)

cmdIndex0 = cmdGroup.AddCommandItem("CreateCube", -1, "Create a cube", "Create cube", 0, "CreateCube", "", 0)
cmdIndex1 = cmdGroup.AddCommandItem("Show PMP", -1, "Show PMP", "Show PMP", 2, "ShowPMP", "PMPEnable", 2)

cmdGroup.HasToolBar = True
cmdGroup.HasMenu = True
cmdGroup.Activate()

For Each docType As Integer In docTypes
    Dim cmdTab As ICommandTab = iCmdMgr.GetCommandTab(docType, Title)
    Dim bResult As Boolean

    If cmdTab Is Nothing Then
        cmdTab = iCmdMgr.AddCommandTab(docType, Title)

        Dim cmdBox As CommandTabBox = cmdTab.AddCommandTabBox

        Dim cmdIDs(3) As Integer
        Dim TextType(3) As Integer

        cmdIDs(0) = cmdGroup.CommandID(cmdIndex0)
        TextType(0) = swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal

        cmdIDs(1) = cmdGroup.CommandID(cmdIndex1)
        TextType(1) = swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal

        cmdIDs(2) = cmdGroup.ToolbarId
        TextType(2) = swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal

        bResult = cmdBox.AddCommands(cmdIDs, TextType)

        Dim cmdBox1 As CommandTabBox = cmdTab.AddCommandTabBox()
        ReDim cmdIDs(1)
        ReDim TextType(1)

        cmdIDs(0) = cmdGroup.ToolbarId
        TextType(0) = swCommandTabButtonTextDisplay_e.swCommandTabButton_TextBelow

        bResult = cmdBox1.AddCommands(cmdIDs, TextType)

        cmdTab.AddSeparator(cmdBox1, cmdGroup.ToolbarId)
    End If
Next

' create a third-party icon in the context-sensitive menus of faces in parts
frame = SwApp.Frame()
resultCode = frame.AddMenuPopupIcon2(swDocumentTypes_e.swDocPART, swSelectType_e.swSelfFACES, "third-party context-sensitive", addinID, "CSCallbackFunction", "CSEnable", "", cmdGroup.SmallMainIcon)

' create and register the shortcut menu
registerID = SwApp.RegisterThirdPartyPopupMenu()
' add a menu break at the top of the shortcut menu
resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, CInt(swDocumentTypes_e.swDocPART), "Menu Break", addinID, "", "", "", CInt(swMenuItemType_e.swMenuItemType_Break))
' add a couple of items to the shortcut menu
resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, CInt(swDocumentTypes_e.swDocPART), "Test1", addinID, "TestCallback", "EnableTest", "", "Test1", cmdGroup.SmallMainIcon, CInt(swMenuItemType_e.swMenuItemType_Default))
resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, CInt(swDocumentTypes_e.swDocPART), "Test4", addinID, "TestCallback", "EnableTest", "", "Test4", cmdGroup.SmallMainIcon, CInt(swMenuItemType_e.swMenuItemType_Default))
' add a separator bar to the shortcut menu
resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, CInt(swDocumentTypes_e.swDocPART), "separator", addinID, "", "", "", CInt(swMenuItemType_e.swMenuItemType_Separator))
' add another item to the shortcut menu
resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, CInt(swDocumentTypes_e.swDocPART), "Test5", addinID, "TestCallback", "EnableTest", "", "Test5", cmdGroup.SmallMainIcon, CInt(swMenuItemType_e.swMenuItemType_Default))

' add an icon to a menu bar of the shortcut menu
resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, CInt(swDocumentTypes_e.swDocPART), "", addinID, "TestCallback", "EnableTest", "", "NoOp", cmdGroup.SmallMainIcon, CInt(swMenuItemType_e.swMenuItemType_Default))

' remove an icon from the menu bar of the shortcut menu
' resultCode = SwApp.RemoveItemFromThirdPartyPopupMenu(registerID, CInt(swDocumentTypes_e.swDocPART), "", 1)
thisAssembly = Nothing
iBmp.Dispose()
End Sub

Public Sub RemoveCommandMgr()
    iCmdMgr.RemoveCommandGroup(1)
End Sub

Function AddPMP() As Boolean
    ppage = New UserPMPPage
    ppage.Init(iSwApp, Me)
End Function

Function RemovePMP() As Boolean
    ppage = Nothing
End Function
#End Region

```

```

#Region "Event Methods"
Sub AttachEventHandlers()
    AttachSWEvents()

    'Listen for events on all currently open docs
    AttachEventsToAllDocuments()
End Sub

Sub DetachEventHandlers()
    DetachSWEvents()

    'Close events on all currently open docs
    Dim docHandler As DocumentEventHandler
    Dim key As ModelDoc2
    Dim numKeys As Integer
    numKeys = openDocs.Count
    If numKeys > 0 Then
        Dim keys() As Object = New Object(numKeys - 1) {}

        'Remove all document event handlers
        openDocs.Keys.CopyTo(keys, 0)
        For Each key In keys
            docHandler = openDocs.Item(key)
            docHandler.DetachEventHandlers() 'This also removes the pair from the hash
            docHandler = Nothing
            key = Nothing
        Next
    End If
End Sub

Sub AttachSWEvents()
    AddHandler iSwApp.ActiveDocChangeNotify, AddressOf Me.SldWorks_ActiveDocChangeNotify
    AddHandler iSwApp.DocumentLoadNotify2, AddressOf Me.SldWorks_DocumentLoadNotify2
    AddHandler iSwApp.FileNewNotify2, AddressOf Me.SldWorks_FileNewNotify2
    AddHandler iSwApp.ActiveModelDocChangeNotify, AddressOf Me.SldWorks_ActiveModelDocChangeNotify
    AddHandler iSwApp.FileOpenPostNotify, AddressOf Me.SldWorks_FileOpenPostNotify
    AddHandler iSwApp.InterfaceBrightnessThemeChangeNotify, AddressOf Me.SldWorks_InterfaceBrightnessChangeThemeNotify

End Sub

Sub DetachSWEvents()
    RemoveHandler iSwApp.ActiveDocChangeNotify, AddressOf Me.SldWorks_ActiveDocChangeNotify
    RemoveHandler iSwApp.DocumentLoadNotify2, AddressOf Me.SldWorks_DocumentLoadNotify2
    RemoveHandler iSwApp.FileNewNotify2, AddressOf Me.SldWorks_FileNewNotify2
    RemoveHandler iSwApp.ActiveModelDocChangeNotify, AddressOf Me.SldWorks_ActiveModelDocChangeNotify
    RemoveHandler iSwApp.FileOpenPostNotify, AddressOf Me.SldWorks_FileOpenPostNotify
    RemoveHandler iSwApp.InterfaceBrightnessThemeChangeNotify, AddressOf Me.SldWorks_InterfaceBrightnessChangeThemeNotify

End Sub

Sub AttachEventsToAllDocuments()
    Dim modDoc As ModelDoc2
    modDoc = iSwApp.GetFirstDocument()
    While Not modDoc Is Nothing
        If Not openDocs.Contains(modDoc) Then
            AttachModelDocEventHandler(modDoc)
        End If
        modDoc = modDoc.GetNext()
    End While
End Sub

Function AttachModelDocEventHandler(ByVal modDoc As ModelDoc2) As Boolean
    If modDoc Is Nothing Then
        Return False
    End If
    Dim docHandler As DocumentEventHandler = Nothing

    If Not openDocs.Contains(modDoc) Then
        Select Case modDoc.GetType
            Case swDocumentTypes_e.swDocPART
                docHandler = New PartEventHandler()
            Case swDocumentTypes_e.swDocASSEMBLY
                docHandler = New AssemblyEventHandler()
            Case swDocumentTypes_e.swDocDRAWING
                docHandler = New DrawingEventHandler()
        End Select

        docHandler.Init(iSwApp, Me, modDoc)
        docHandler.AttachEventHandlers()
        openDocs.Add(modDoc, docHandler)
    End If
End Function

Sub DetachModelDocEventHandler(ByVal modDoc As ModelDoc2)
    Dim docHandler As DocumentEventHandler
    docHandler = openDocs.Item(modDoc)
    openDocs.Remove(modDoc)
    modDoc = Nothing
    docHandler = Nothing
End Sub
#End Region

#Region "Event Handlers"
Function SldWorks_ActiveDocChangeNotify() As Integer
    'TODO: Add your implementation here
End Function

```

```

Function SldWorks_DocumentLoadNotify2(ByVal docTitle As String, ByVal docPath As String) As Integer
End Function

Function SldWorks_FileNewNotify2(ByVal newDoc As Object, ByVal doctype As Integer, ByVal templateName As String) As Integer
    AttachEventsToAllDocuments()
End Function

Function SldWorks_ActiveModelDocChangeNotify() As Integer
    'TODO: Add your implementation here
End Function

Function SldWorks_FileOpenPostNotify(ByVal FileName As String) As Integer
    AttachEventsToAllDocuments()
End Function

Function SldWorks_InterfaceBrightnessChangeThemeNotify(ByVal Theme As Integer, ByRef Colors as Object) As Integer
    MsgBox("Interface brightness theme changed to (0 = light, 1 = medium, 2 = dark): " & Theme)
    Dim colorsArray() As Integer
    colorsArray = Colors
    MsgBox("FeatureManager design tree color: " & colorsArray(0))
End Function
#End Region

#Region "UI Callbacks"
Sub CreateCube()

    'make sure we have a part open
    Dim partTemplate As String

    Dim featMan As FeatureManager

    partTemplate = iSwApp.GetUserPreferenceStringValue(swUserPreferenceStringValue_e.swDefaultTemplatePart)
    model = iSwApp.NewDocument(partTemplate, swDwgPaperSizes_e.swDwgPaperA2size, 0.0, 0.0)

    model.InsertSketch2(True)
    model.SketchRectangle(0, 0, 0, 0.1, 0.1, 0.1, False)

    'Extrude the sketch
    featMan = model.FeatureManager
    featMan.FeatureExtrusion(True, _
        False, False, _
        swEndConditions_e.swEndCondBlind, swEndConditions_e.swEndCondBlind, _
        0.1, 0.0, _
        False, False, _
        False, False, _
        0.0, 0.0, _
        False, False, _
        False, False, _
        True, _
        False, False)

End Sub
Sub CSCallbackFunction()
    resultCode = SwApp.ShowThirdPartyPopupMenu(registerID, 500, 500)
End Sub
Function CSEnable() As Integer
    If iSwApp.ActiveDoc Is Nothing Then
        CSEnable = 0
    Else
        CSEnable = 1
    End If
End Function
Sub ShowPMP()
    ppage.Show()
End Sub

Function PMPEnable() As Integer
    If iSwApp.ActiveDoc Is Nothing Then
        PMPEnable = 0
    Else
        PMPEnable = 1
    End If
End Function
Sub TestCallback()
    System.Diagnostics.Debug.Print("Test callback")
End Sub
Function EnableTest() As Integer
    If iSwApp.ActiveDoc Is Nothing Then
        EnableTest = 0
    Else
        EnableTest = 1
    End If
End Function
#End Region
End Class

```

Add Attribute to Feature and Include in Library Feature Example (C#)

This example shows how to add an attribute to a feature and include that attribute with the feature if the feature is saved as a library feature. This example also includes instructions on how to verify that the attribute is included on each instance of the library feature.

```
//-----
// Preconditions:
// 1. Open a new part document.
// 2. Sketch a rectangle and extrude it.
// 3. Sketch a straight slot that fits on a face of
//    of the just-created extrude and cut-extrude the slot.
//
// NOTE: The cut-extrude must be named Cut-Extrude1.
//
// 4. Open the Immediate window.
//
// Postconditions:
// 1. Adds the cut-extrude feature to the part document with
//    an attribute of TestAttribute, which is visible in the
//    FeatureManager design tree.
// 2. Examine the Immediate window.
// 3. To verify that the attribute is included in a library feature:
//    a. Click Design Library on the Task Pane.
//       1. Click Add to Library.
//       2. Click Cut-Extrude1 in the flyout FeatureManager design tree.
//       3. Type a file name for the library feature.
//       4. Select the folder where to save the library feature.
//       5. Click OK.
//    b. Open an existing model document and drag-and-drop the
//       just-created library feature on the model and click OK.
//    c. Expand the just-dropped library feature in the FeatureManager
//       design tree to verify that Cut-Extrude1 and TestAttribute appear
//       beneath the just-dropped library feature in the FeatureManager
//       design tree.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Diagnostics;
using System;

namespace NameAttributeCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            Feature swFeature = default(Feature);
            SolidWorks.Interop.sldworks.Attribute swAttribute = default(SolidWorks.Interop.sldworks.Attribute);
            AttributeDef swAttributeDef = default(AttributeDef);
            Face2 swFace = default(Face2);
            Parameter swParameter = default(Parameter);
            Object[] Faces = null;
            bool @bool = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = swModel.Extension;
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;

            // Create attribute
            swAttributeDef = (AttributeDef)swApp.DefineAttribute("TestPropagationOfAttribute");
            @bool = swAttributeDef.AddParameter("TestAttribute", (int)swParamType_e.swParamTypeDouble, 2.0, 0);
            @bool = swAttributeDef.Register();

            // Select the feature to which to add the attribute
            @bool = swModelDocExt.SelectByID("Cut-Extrude1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            swFeature = (Feature)swSelectionMgr.GetSelectedObject6(1, -1);
            Debug.Print("Name of feature to which to add attribute: " + swFeature.Name);

            // Add the attribute to one of the feature's faces
            Faces = (Object[])swFeature.GetFaces();
            swFace = (Face2)Faces[0];
            swAttribute = swAttributeDef.CreateInstance5(swModel, swFace, "TestAttribute", 0, (int)swInConfigurationOpts_e.swAllConfiguration);
            swAttribute.IncludeInLibraryFeature = true;
            Debug.Print("Include attribute in library feature? " + swAttribute.IncludeInLibraryFeature);
            Debug.Print("Name of attribute: " + swAttribute.GetName());

            // Get name of parameter
            swParameter = (
                Parameter)swAttribute.GetParameter("TestAttribute");
            Debug.Print("Parameter name: " + swParameter.GetName());

            swModel.ForceRebuild3(false);
        }
    }
}

public SldWorks swApp;
```

Add Attribute to Feature and Include In Library Feature Example (VBA)

This example shows how to add an attribute to a feature and include that attribute with the feature if the feature is saved as a library feature. This example also includes instructions on how to verify that the attribute is included on each instance of the library feature.

```

'-----
' Preconditions:
' 1. Open a new part document.
' 2. Sketch a rectangle and extrude it.
' 3. Sketch a straight slot that fits on a face of
'    of the just-created extrude and cut-extrude the slot.
'
'    NOTE: The cut-extrude feature must be named Cut-Extrude1.
'
' 4. Open the Immediate window.
'
' Postconditions:
' 1. Adds the cut-extrude feature to the part document with
'    an attribute of TestAttribute, which is visible in the
'    FeatureManager design tree.
' 2. Examine the Immediate window.
' 3. To verify that the attribute is included in a library feature:
'    a. Click Design Library on the Task Pane.
'       1. Click Add to Library.
'       2. Click Cut-Extrude1 in the flyout FeatureManager design tree.
'       3. Type a file name for the library feature.
'       4. Select the folder where to save the library feature.
'       5. Click OK.
'    b. Open an existing model document and drag-and-drop the
'       just-created library feature on the model and click OK.
'    c. Expand the just-dropped library feature in the FeatureManager
'       design tree to verify that Cut-Extrude1 and TestAttribute appear
'       beneath the just-dropped library feature in the FeatureManager
'       design tree.
'-----

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swFeature As SldWorks.Feature
Dim swAttribute As SldWorks.Attribute
Dim swAttributeDef As SldWorks.AttributeDef
Dim swFace As SldWorks.Face2
Dim swParameter as SldWorks.Parameter
Dim Faces As Variant
Dim bool As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension
    Set swSelectionMgr = swModel.SelectionManager

    ' Create attribute
    Set swAttributeDef = swApp.DefineAttribute("TestPropagationOfAttribute")
    bool = swAttributeDef.AddParameter("TestAttribute", swParamTypeDouble, 2#, 0)
    bool = swAttributeDef.Register

    ' Select the feature to which to add the attribute
    bool = swModelDocExt.SelectByID2("Cut-Extrude1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    Set swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
    Debug.Print ("Name of feature to which to add attribute: " & swFeature.Name)

    ' Add the attribute to one of the feature's faces
    Faces = swFeature.GetFaces
    Set swFace = Faces(0)
    Set swAttribute = swAttributeDef.CreateInstance5(swModel, swFace, "TestAttribute", 0, swAllConfiguration)
    swAttribute.IncludeInLibraryFeature = True
    Debug.Print ("Include attribute in library feature? " & swAttribute.IncludeInLibraryFeature)
    Debug.Print ("Name of attribute: " & swAttribute.GetName)

    ' Add the attribute to one of the feature's faces
    Set swParameter = swAttribute.GetParameter("TestAttribute")
    Debug.Print ("Name of parameter: " & swParameter.GetName)

    swModel.ForceRebuild3 (False)

End Sub

```

Add Attribute to Feature and Include in Library Feature Example (VB.NET)

This example shows how to add an attribute to a feature and include that attribute with the feature if the feature is saved as a library feature. This example also includes instructions on how to verify that the attribute is included on each instance of the library feature.

```

-----
Preconditions:
1. Open a new part document.
2. Sketch a rectangle and extrude it.
3. Sketch a straight slot that fits on a face of
   of the just-created extrude and cut-extrude the slot.

   NOTE: The cut-extrude feature must be named Cut-Extrude1.

4. Open the Immediate window.

Postconditions:
1. Adds the cut-extrude feature to the part document with
   an attribute of TestAttribute, which is visible in the
   FeatureManager design tree.
2. Examine the Immediate window.
3. To verify that the attribute is included in a library feature:
   a. Click Design Library on the Task Pane.
      1. Click Add to Library.
      2. Click Cut-Extrude1 in the flyout FeatureManager design tree.
      3. Type a file name for the library feature.
      4. Select the folder where to save the library feature.
      5. Click OK.
   b. Open an existing model document and drag-and-drop the
      just-created library feature on the model and click OK.
   c. Expand the just-dropped library feature in the FeatureManager
      design tree to verify that Cut-Extrude1 and TestAttribute appear
      beneath the just-dropped library feature in the FeatureManager
      design tree.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Diagnostics
Imports System

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelectionMgr As SelectionMgr
        Dim swFeature As Feature
        Dim swAttribute As SolidWorks.Interop.sldworks.Attribute
        Dim swAttributeDef As AttributeDef
        Dim swFace As Face2
        Dim swParameter As Parameter
        Dim Faces As Object
        Dim bool As Boolean

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension
        swSelectionMgr = swModel.SelectionManager

        ' Create attribute
        swAttributeDef = swApp.DefineAttribute("TestPropagationOfAttribute")
        bool = swAttributeDef.AddParameter("TestAttribute", swParamType_e.swParamTypeDouble, 2.0#, 0)
        bool = swAttributeDef.Register

        ' Select the feature to which to add the attribute
        bool = swModelDocExt.SelectByID2("Cut-Extrude1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
        Debug.Print("Name of feature to which to add attribute: " & swFeature.Name)

        ' Add the attribute to one of the feature's faces
        Faces = swFeature.GetFaces
        swFace = Faces(0)
        swAttribute = swAttributeDef.CreateInstance5(swModel, swFace, "TestAttribute", 0, swInConfiguration0pts_e.swAllConfiguration)
        swAttribute.IncludeInLibraryFeature = True
        Debug.Print("Include attribute in library feature? " & swAttribute.IncludeInLibraryFeature)
        Debug.Print("Name of attribute: " & swAttribute.GetName)

        ' Get name of parameter
        swParameter = swAttribute.GetParameter("TestAttribute")
        Debug.Print("Name of parameter: " & swParameter.GetName)

        swModel.ForceRebuild3(False)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Add Auto Balloons to Drawing Example (C#)

This example shows how to automatically add BOM balloons to a drawing view.

```
//-----
// Preconditions: Open a drawing with a bill of materials (BOM) table.
//
// Postconditions: BOM balloons are added to the view.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace AutoBalloon_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        DrawingDoc Draw;
        object vNotes;
        AutoBalloonOptions autoballoonParams;

        bool boolstatus;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;
            Draw = (DrawingDoc)Part;
            boolstatus = Draw.ActivateView("Drawing View1");
            boolstatus = Part.Extension.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, false, 0, null, 0);

            autoballoonParams = Draw.CreateAutoBalloonOptions();
            autoballoonParams.Layout = (int)swBalloonLayoutType_e.swDetailingBalloonLayout_Square;
            autoballoonParams.ReverseDirection = false;
            autoballoonParams.IgnoreMultiple = true;
            autoballoonParams.InsertMagneticLine = true;
            autoballoonParams.LeaderAttachmentToFaces = true;
            autoballoonParams.Style = (int)swBalloonStyle_e.swBS_Circular;
            autoballoonParams.Size = (int)swBalloonFit_e.swBF_5Chars;
            autoballoonParams.UpperTextContent = (int)swBalloonTextContent_e.swBalloonTextItemNumber;
            autoballoonParams.Layername = "-None-";
            autoballoonParams.ItemNumberStart = 1;
            autoballoonParams.ItemNumberIncrement = 1;
            autoballoonParams.ItemOrder = (int)swBalloonItemNumbersOrder_e.swBalloonItemNumbers_DoNotChangeItemNumbers;
            autoballoonParams.EditBalloons = true;
            autoballoonParams.EditBalloonOption = (int)swEditBalloonOption_e.swEditBalloonOption_Resequence;

            vNotes = Draw.AutoBalloon5(autoballoonParams);
        }

        public SldWorks swApp;
    }
}
```

Add Auto Balloons to Drawing Example (VBA)

This example shows how to automatically add BOM balloons to a drawing view.

```
'-----  
' Preconditions: Open a drawing with a bill of materials (BOM) table.  
'  
' Postconditions: The specified BOM balloons are added to the view.  
'-----  
Dim swApp As SldWorks.SldWorks  
Dim Part As SldWorks.ModelDoc2  
Dim vNotes As Variant  
Dim autoballoonParams As SldWorks.AutoBalloonOptions  
Dim boolstatus As Boolean  
Option Explicit  
Sub main()  
    Set swApp = Application.SldWorks  
    Set Part = swApp.ActiveDoc  
    boolstatus = Part.ActivateView("Drawing View1")  
    boolstatus = Part.Extension.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)  
  
    Set autoballoonParams = Part.CreateAutoBalloonOptions()  
    autoballoonParams.Layout = swDetailingBalloonLayout_Square  
    autoballoonParams.ReverseDirection = False  
    autoballoonParams.IgnoreMultiple = True  
    autoballoonParams.InsertMagneticLine = True  
    autoballoonParams.LeaderAttachmentToFaces = True  
    autoballoonParams.Style = swBS_Circular  
    autoballoonParams.Size = swBF_5Chars  
    autoballoonParams.UpperTextContent = swBalloonTextItemNumber  
    autoballoonParams.Layername = "-None-"  
    autoballoonParams.ItemNumberStart = 1  
    autoballoonParams.ItemNumberIncrement = 1  
    autoballoonParams.ItemOrder = swBalloonItemNumbers_DoNotChangeItemNumbers  
    autoballoonParams.EditBalloons = True  
    autoballoonParams.EditBalloonOption = swEditBalloonOption_Resequence  
  
    vNotes = Part.AutoBalloon5(autoballoonParams)  
  
End Sub
```

Add Auto Balloons to Drawing Example (VB.NET)

This example shows how to automatically add BOM balloons to a drawing view.

```
'-----
' Preconditions: Open a drawing with a bill of materials (BOM) table.
' Postconditions: BOM balloons are added to the view.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim Part As ModelDoc2
    Dim vNotes As Object
    Dim autoballoonParams As AutoBalloonOptions
    Dim boolstatus As Boolean
```

```
    Sub main()
```

```
        Part = swApp.ActiveDoc
        boolstatus = Part.ActivateView("Drawing View1")
        boolstatus = Part.Extension.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)
```

```
        autoballoonParams = Part.CreateAutoBalloonOptions()
        autoballoonParams.Layout = swBalloonLayoutType_e.swDetailingBalloonLayout_Square
        autoballoonParams.ReverseDirection = False
        autoballoonParams.IgnoreMultiple = True
        autoballoonParams.InsertMagneticLine = True
        autoballoonParams.LeaderAttachmentToFaces = True
        autoballoonParams.Style = swBalloonStyle_e.swBS_Circular
        autoballoonParams.Size = swBalloonFit_e.swBF_5Chars
        autoballoonParams.UpperTextContent = swBalloonTextContent_e.swBalloonTextItemNumber
        autoballoonParams.Layername = "-None-"
        autoballoonParams.ItemNumberStart = 1
        autoballoonParams.ItemNumberIncrement = 1
        autoballoonParams.ItemOrder = swBalloonItemNumbersOrder_e.swBalloonItemNumbers_DoNotChangeItemNumbers
        autoballoonParams.EditBalloons = True
        autoballoonParams.EditBalloonOption = swEditBalloonOption_Resequence
```

```
        vNotes = Part.AutoBalloon5(autoballoonParams)
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Add Balloon to Stacked Balloon Example (C#)

This example shows how to create a stacked balloon and add a balloon to the stacked balloon.

```
//-----
// Preconditions: Verify that the specified assembly
// document to open exists.
//
// Postconditions:
// 1. Opens the specified assembly document.
// 2. Selects a face on the Part4 component.
// 3. Creates a stacked balloon showing the item number
//    of Part4 and inserts that stacked balloon on the selected face.
// 4. Adds a balloon, which shows the number of seed and derived Part4
//    components in the the assembly, to the stacked balloon.
// 5. Click and drag the stacked balloon upward in the graphics area.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace StackedBalloonsCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            StackedBalloonOptions swStackedBalloons = default(StackedBalloonOptions);
            Note swNote = default(Note);
            BalloonStack swBalloonStack = default(BalloonStack);
            Note swNote2 = default(Note);
            string fileName = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\assem2.sldasm";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref e
            rrors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            swModel.ViewZoomtofit2();

            status = swModelDocExt.SelectByID2("", "FACE", -0.311973500811291, 0.0329693343254007, 0.00999999999999991, false, 0, null, 0);
            swStackedBalloons = swModelDocExt.CreateStackedBalloonOptions();
            swStackedBalloons.BalloonsPerLine = 10;
            swStackedBalloons.StackDirection = (int)swStackedBalloonDirection_e.swStackedBalloonDir_Right;
            swStackedBalloons.Style = (int)(int)swBalloonStyle_e.swBS_Circular;
            swStackedBalloons.Size = (int)swBalloonFit_e.swBF_2Chars;
            swStackedBalloons.UpperTextContent = (int)swBalloonTextContent_e.swBalloonTextItemNumber;
            swStackedBalloons.UpperText = "\\\"",
            swStackedBalloons.ShowQuantity = false;
            swStackedBalloons.QuantityPlacement = (int)swBalloonQuantityPlacement_e.swBalloonQuantityPlacement_Top;
            swStackedBalloons.QuantityDenotationText = "X";
            swStackedBalloons.QuantityOverride = false;
            swNote = (Note)swModelDocExt.InsertStackedBalloon2(swStackedBalloons);
            status = swModelDocExt.SelectByID2("", "FACE", -0.292957708052256, 0.0316192505406434, 0.01000000000000477, false, 0, null, 0);

            if (swNote.IsStackedBalloon())
            {
                if ((swNote != null))
                {
                    swBalloonStack = (BalloonStack)swNote.GetBalloonStack();
                    swNote2 = (Note)swBalloonStack.AddTo((int)swBalloonTextContent_e.swBalloonTextQuantity, "12", (int)swBalloonTextContent_e.swBalloonTex
                    tQuantity, "");
                }
            }

            /// <summary>
            /// The SldWorks swApp variable is pre-assigned for you.
            /// </summary>
            public SldWorks swApp;
        }
    }
}
```

Add Balloon to Stacked Balloon Example (VBA)

This example shows how to create a stacked balloon and add a balloon to the stacked balloon.

```
'-----
' Preconditions: Verify that the specified assembly
' document to open exists.
'
' Postconditions:
' 1. Opens the specified assembly document.
' 2. Selects a face on the Part4 component.
' 3. Creates a stacked balloon showing the item number
'    of Part4 and inserts that stacked balloon on the selected face.
' 4. Adds a balloon, which shows the number of seed and derived Part4
'    components in the the assembly, to the stacked balloon.
' 5. Click and drag the stacked balloon upward in the graphics area.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swStackedBalloons As SldWorks.StackedBalloonOptions
Dim swNote As SldWorks.Note
Dim swBalloonStack As SldWorks.BalloonStack
Dim swNote2 As SldWorks.Note
Dim fileName As String
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
```

Sub main()

```
Set swApp = Application.SldWorks
fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\assem2.sldasm"
Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
Set swModelDocExt = swModel.Extension
```

swModel.ViewZoomtofit2

```
status = swModelDocExt.SelectByID("", "FACE", -0.311973500811291, 3.29693343254007E-02, 9.99999999999991E-03, False, 0, Nothing, 0)
Set swStackedBalloons = swModelDocExt.CreateStackedBalloonOptions()
swStackedBalloons.BalloonsPerLine = 10
swStackedBalloons.StackDirection = swStackedBalloonDirection_e.swStackedBalloonDir_Right
swStackedBalloons.Style = swBalloonStyle_e.swBS_Circular
swStackedBalloons.Size = swBalloonFit_e.swBF_2Chars
swStackedBalloons.UpperTextContent = swBalloonTextContent_e.swBalloonTextItemNumber
swStackedBalloons.UpperText = ""
swStackedBalloons.ShowQuantity = False
swStackedBalloons.QuantityPlacement = swBalloonQuantityPlacement_e.swBalloonQuantityPlacement_Top
swStackedBalloons.QuantityDenotationText = "X"
swStackedBalloons.QuantityOverride = False
Set swNote = swModelDocExt.InsertStackedBalloon2(swStackedBalloons)
status = swModelDocExt.SelectByID("", "FACE", -0.292957708052256, 3.16192505406434E-02, 1.00000000000477E-02, False, 0, Nothing, 0)
```

```
If swNote.IsStackedBalloon Then
If Not swNote Is Nothing Then
Set swBalloonStack = swNote.GetBalloonStack()
Set swNote2 = swBalloonStack.AddTo(swBalloonTextContent_e.swBalloonTextQuantity, "12", swBalloonTextContent_e.swBalloonTextQuantity, "")
End If
End If
```

End Sub

Add Balloon to Stacked Balloon Example_VBNET (Topic)

SOLIDWORKS API Help

Add Balloon to Stacked Balloon Example (VB.NET)

This example shows how to create a stacked balloon and add a balloon to the stacked balloon.

```
-----
Preconditions: Verify that the specified assembly
document to open exists.

Postconditions:
1. Opens the specified assembly document.
2. Selects a face on the Part4 component.
3. Creates a stacked balloon showing the item number
of Part4 and inserts that stacked balloon on the selected face.
4. Adds a balloon, which shows the number of seed and derived Part4
components in the the assembly, to the stacked balloon.
5. Click and drag the stacked balloon upward in the graphics area.
-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swStackedBalloons As StackedBalloonOptions
        Dim swNote As Note
        Dim swBalloonStack As BalloonStack
        Dim swNote2 As Note
        Dim fileName As String
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS_2018\samples\tutorial\api\assem2.sldasm"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension

        swModel.ViewZoomtofit2()

        status = swModelDocExt.SelectByID("", "FACE", -0.311973500811291, 0.0329693343254007, 0.00999999999999991, False, 0, Nothing, 0)
        swStackedBalloons = swModelDocExt.CreateStackedBalloonOptions()
        swStackedBalloons.BalloonsPerLine = 10
        swStackedBalloons.StackDirection = swStackedBalloonDirection_e.swStackedBalloonDir_Right
        swStackedBalloons.Style = swBalloonStyle_e.swBS_Circular
        swStackedBalloons.Size = swBalloonFit_e.swBF_2Chars
        swStackedBalloons.UpperTextContent = swBalloonTextContent_e.swBalloonTextItemNumber
        swStackedBalloons.UpperText = ""
        swStackedBalloons.ShowQuantity = False
        swStackedBalloons.QuantityPlacement = swBalloonQuantityPlacement_e.swBalloonQuantityPlacement_Top
        swStackedBalloons.QuantityDenotationText = "X"
        swStackedBalloons.QuantityOverride = False
        swNote = swModelDocExt.InsertStackedBalloon2(swStackedBalloons)
        status = swModelDocExt.SelectByID("", "FACE", -0.292957708052256, 0.0316192505406434, 0.01000000000000477, False, 0, Nothing, 0)

        If swNote.IsStackedBalloon Then
            If Not swNote.Is Nothing Then
                swBalloonStack = swNote.GetBalloonStack()
                swNote2 = swBalloonStack.AddTo(swBalloonTextContent_e.swBalloonTextQuantity, "12", swBalloonTextContent_e.swBalloonTextQuantity, "")
            End If
        End If

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks
```

```
End Class
```

Add Button to Context-sensitive Menu Example (C#)

This example shows how to add:

- button to a face's context-sensitive menu in a part.
- shortcut menu for that button.

```
//-----
// Preconditions:
// 1. Verify that the SOLIDWORKS API SDK add-in templates for your version
// of Microsoft Visual Studio are installed.
// 2. In Microsoft Visual Studio, create a C# project using the
// SwSharpAddin template.
// 3. Name the project AddMenuPopupIconCSharp.
// 4. Copy and paste SwAddin into SwAddin.cs of your project.
// 5. Click Project > AddMenuPopupIconCSharp Properties > Debug.
// 6. Select Start external program and type the pathname to your
// SOLIDWORKS Design executable.
// 7. Replace the text shown for the imageList array elements
// and smallMainIcon with the pathnames to your image files.
// 8. Press F5 to start debugging this add-in.
//
// Postconditions:
// 1. In SOLIDWORKS Design, open a part.
// 2. Right-click a face of the part.
// 3. Note the size of the button added by this add-in to the
// context-sensitive menu.
// 4. Click the button added by this add-in to display the shortcut menu.
// 5. Exit SOLIDWORKS Design.
// 6. Change the size of the text and other items on your computer.
// For example, in Windows 7:
// a. Click Start > Control Panel > Appearance and
// Personalization > Display.
// b. Select a different size.
// c. Click Apply.
// d. Click Log off now.
// e. Log back in.
// 7. Start SOLIDWORKS Design.
// 8. Repeat steps 1 - 3.
//-----
//SwAddin
using System;
using System.Runtime.InteropServices;
using System.Collections;
using System.Reflection;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swpublished;
using SolidWorks.Interop.swconst;
using SolidWorksTools;
using SolidWorksTools.File;
using System.Collections.Generic;
using System.Diagnostics;

namespace AddMenuPopupCSharp
{
    /// <summary>
    /// Summary description for AddMenuPopupCSharp.
    /// </summary>
    [Guid("7d3de7ec-7f66-4827-af80-1411b18c7dfa"), ComVisible(true)]
    [SwAddin(
        Description = "AddMenuPopupCSharp description",
        Title = "AddMenuPopupCSharp",
        LoadAtStartup = true
    )]
    public class SwAddin : ISwAddin
    {
        #region Local Variables
        SldWorks iSwApp;
        int addinID;
        BitmapHandler iBmp;
        ModelDoc2 model;
        IFrame frame;
        bool resultCode;
        int registerID;

        #region Event Handler Variables
        Hashtable openDocs = new Hashtable();
        SolidWorks.Interop.sldworks.SldWorks SwEventPtr = null;
        #endregion

        // Public Properties
        public ISldWorks SwApp
        {
            get { return iSwApp; }
        }

        public Hashtable OpenDocs
        {
            get { return openDocs; }
        }

        #endregion

        #region SolidWorks Registration
        [ComRegisterFunctionAttribute]
        public static void RegisterFunction(Type t)
        {
            #region Get Custom Attribute: SwAddinAttribute
            SwAddinAttribute SwAttr = null;
            Type type = typeof(SwAddin);

            foreach (System.Attribute attr in type.GetCustomAttributes(false))
            {
                if (attr is SwAddinAttribute)
                {
                    SwAttr = attr as SwAddinAttribute;
                    break;
                }
            }
        }
    }
}
```

```

    }

    #endregion

    try
    {
        Microsoft.Win32.RegistryKey hklm = Microsoft.Win32.Registry.LocalMachine;
        Microsoft.Win32.RegistryKey hkcu = Microsoft.Win32.Registry.CurrentUser;

        string keyname = "SOFTWARE\\SolidWorks\\Addins\\{" + t.GUID.ToString() + "}";
        Microsoft.Win32.RegistryKey addinkey = hklm.CreateSubKey(keyname);
        addinkey.SetValue(null, 0);

        addinkey.SetValue("Description", Swattr.Description);
        addinkey.SetValue("Title", Swattr.Title);

        keyname = "Software\\SolidWorks\\AddInsStartup\\{" + t.GUID.ToString() + "}";
        addinkey = hkcu.CreateSubKey(keyname);
        addinkey.SetValue(null, Convert.ToInt32(Swattr.LoadAtStartup), Microsoft.Win32.RegistryValueKind.DWord);
    }
    catch (System.NullReferenceException nl)
    {
        Console.WriteLine("There was a problem registering this dll: Swattr is null. \\n\\n" + nl.Message + "\\n\\n");
        System.Windows.Forms.MessageBox.Show("There was a problem registering this dll: Swattr is null. \\n\\n" + nl.Message + "\\n\\n");
    }

    catch (System.Exception e)
    {
        Console.WriteLine(e.Message);

        System.Windows.Forms.MessageBox.Show("There was a problem registering the function: \\n\\n" + e.Message + "\\n\\n");
    }
}

[ComUnregisterFunctionAttribute]
public static void UnregisterFunction(Type t)
{
    try
    {
        Microsoft.Win32.RegistryKey hklm = Microsoft.Win32.Registry.LocalMachine;
        Microsoft.Win32.RegistryKey hkcu = Microsoft.Win32.Registry.CurrentUser;

        string keyname = "SOFTWARE\\SolidWorks\\Addins\\{" + t.GUID.ToString() + "}";
        hklm.DeleteSubKey(keyname);

        keyname = "Software\\SolidWorks\\AddInsStartup\\{" + t.GUID.ToString() + "}";
        hkcu.DeleteSubKey(keyname);
    }
    catch (System.NullReferenceException nl)
    {
        Console.WriteLine("There was a problem unregistering this dll: " + nl.Message);
        System.Windows.Forms.MessageBox.Show("There was a problem unregistering this dll: \\n\\n" + nl.Message + "\\n\\n");
    }
    catch (System.Exception e)
    {
        Console.WriteLine("There was a problem unregistering this dll: " + e.Message);
        System.Windows.Forms.MessageBox.Show("There was a problem unregistering this dll: \\n\\n" + e.Message + "\\n\\n");
    }
}

#endregion

#region ISwAddin Implementation
public SwAddin()
{
}

public bool ConnectToSW(object ThisSW, int cookie)
{
    iSwApp = (SldWorks)ThisSW;
    addinID = cookie;

    //Set up callbacks
    iSwApp.SetAddinCallbackInfo2(0, this, addinID);

    #region Set up third-party buttons
    AddThirdPartyIcon();
    #endregion

    #region Set up the Event Handlers
    SwEventPtr = (SldWorks)iSwApp;
    openDocs = new Hashtable();
    AttachEventHandlers();
    #endregion

    return true;
}

public bool DisconnectFromSW()
{
    DetachEventHandlers();

    System.Runtime.InteropServices.Marshal.ReleaseComObject(iSwApp);
    iSwApp = null;
    //The add-in must call GC.Collect() here to retrieve all managed code pointers
    GC.Collect();
    GC.WaitForPendingFinalizers();

    GC.Collect();
    GC.WaitForPendingFinalizers();

    return true;
}

#endregion

#region UI Methods
public void AddThirdPartyIcon()
{
    BitmapHandler iBmp = new BitmapHandler();
    Assembly thisAssembly1 = default(Assembly);

    thisAssembly1 = System.Reflection.Assembly.GetAssembly(this.GetType());

    // Create a button in the context-sensitive menus of faces in parts
    frame = SwApp.Frame();
    string[] imageList = new string[3];
    imageList[0] = "Pathname_to_button_nxn_image";
    imageList[1] = "Pathname_to_button_nnnnn_image";
    imageList[2] = "Pathname_to_button_nnnnnnn_image";
}

```

```

        resultCode = frame.AddMenuPopupIcon3((int)swDocumentTypes_e.swDocPART, (int)swSelectType_e.swSelfFACES, "Third-party context-
sensitive", addinID, "CSCallbackFunction", "CSEnable", "", imageList);

        // Create and register the shortcut menu
        registerID = SwApp.RegisterThirdPartyPopupMenu();
        // Add a menu break at the top of the shortcut menu

        resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, Convert.ToInt32(swDocumentTypes_e.swDocPART), "Menu Break", addinID, "", "", "", "", "",
Convert.ToInt32(swMenuItemType_e.swMenuItemType_Break));
        // Add a couple of items to the shortcut menu
        string smallMainIcon = null;
        smallMainIcon = "Pathname_to_shortcut_button_image";

        resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, Convert.ToInt32(swDocumentTypes_e.swDocPART), "Test1", addinID, "TestCallback", "EnableTe
st", "", "Test1", smallMainIcon, Convert.ToInt32(swMenuItemType_e.swMenuItemType_Default));

        resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, Convert.ToInt32(swDocumentTypes_e.swDocPART), "Test4", addinID, "TestCallback", "EnableTe
st", "", "Test4", smallMainIcon, Convert.ToInt32(swMenuItemType_e.swMenuItemType_Default));
        // Add a separator bar to the shortcut menu

        resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, Convert.ToInt32(swDocumentTypes_e.swDocPART), "separator", addinID, "", "", "", "", "", C
onvert.ToInt32(swMenuItemType_e.swMenuItemType_Separator));
        // Add another item to the shortcut menu

        resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, Convert.ToInt32(swDocumentTypes_e.swDocPART), "Test5", addinID, "TestCallback", "EnableTe
st", "", "Test5", smallMainIcon, Convert.ToInt32(swMenuItemType_e.swMenuItemType_Default));
        // Add a button to a menu bar of the shortcut menu

        resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, Convert.ToInt32(swDocumentTypes_e.swDocPART), "", addinID, "TestCallback", "EnableTest",
"", "NoOp", smallMainIcon, Convert.ToInt32(swMenuItemType_e.swMenuItemType_Default));

        thisAssembly1 = null;
        iBmp.Dispose();
    }
}
#endregion

#region Event Methods
public bool AttachEventHandlers()
{
    AttachSwEvents();
    //Listen for events on all currently open documents
    AttachEventsToAllDocuments();
    return true;
}

private bool AttachSwEvents()
{
    try
    {
        SwEventPtr.ActiveDocChangeNotify += new DSldWorksEvents_ActiveDocChangeNotifyEventHandler(ActiveDocChangeNotify);
        SwEventPtr.DocumentLoadNotify2 += new DSldWorksEvents_DocumentLoadNotify2EventHandler(DocumentLoadNotify2);
        SwEventPtr.FileNewNotify2 += new DSldWorksEvents_FileNewNotify2EventHandler(FileNewNotify2);
        SwEventPtr.ActiveModelDocChangeNotify += new DSldWorksEvents_ActiveModelDocChangeNotifyEventHandler(ActiveModelDocChangeNotify);
        SwEventPtr.FileOpenPostNotify += new DSldWorksEvents_FileOpenPostNotifyEventHandler(FileOpenPostNotify);
        return true;
    }
    catch (Exception e)
    {
        Console.WriteLine(e.Message);
        return false;
    }
}

private bool DetachSwEvents()
{
    try
    {
        SwEventPtr.ActiveDocChangeNotify -= new DSldWorksEvents_ActiveDocChangeNotifyEventHandler(ActiveDocChangeNotify);
        SwEventPtr.DocumentLoadNotify2 -= new DSldWorksEvents_DocumentLoadNotify2EventHandler(DocumentLoadNotify2);
        SwEventPtr.FileNewNotify2 -= new DSldWorksEvents_FileNewNotify2EventHandler(FileNewNotify2);
        SwEventPtr.ActiveModelDocChangeNotify -= new DSldWorksEvents_ActiveModelDocChangeNotifyEventHandler(ActiveModelDocChangeNotify);
        SwEventPtr.FileOpenPostNotify -= new DSldWorksEvents_FileOpenPostNotifyEventHandler(FileOpenPostNotify);
        return true;
    }
    catch (Exception e)
    {
        Console.WriteLine(e.Message);
        return false;
    }
}

public void AttachEventsToAllDocuments()
{
    ModelDoc2 modDoc = (ModelDoc2)iSwApp.GetFirstDocument();
    while (modDoc != null)
    {
        if (!openDocs.Contains(modDoc))
        {
            AttachModelDocEventHandler(modDoc);
        }
        else if (openDocs.Contains(modDoc))
        {
            bool connected = false;
            DocumentEventHandler docHandler = (DocumentEventHandler)openDocs[modDoc];
            if (docHandler != null)
            {
                connected = docHandler.ConnectModelViews();
            }
        }

        modDoc = (ModelDoc2)modDoc.GetNext();
    }
}

public bool AttachModelDocEventHandler(ModelDoc2 modDoc)
{
    if (modDoc == null)
        return false;

    DocumentEventHandler docHandler = null;

    if (!openDocs.Contains(modDoc))
    {
        switch (modDoc.GetType())
        {
            case (int)swDocumentTypes_e.swDocPART:
                {

```

```

        docHandler = new PartEventHandler(modDoc, this);
        break;
    }
    case (int)swDocumentTypes_e.swDocASSEMBLY:
    {
        docHandler = new AssemblyEventHandler(modDoc, this);
        break;
    }
    case (int)swDocumentTypes_e.swDocDRAWING:
    {
        docHandler = new DrawingEventHandler(modDoc, this);
        break;
    }
    default:
    {
        //Unsupported document type
        return false;
    }
}
docHandler.AttachEventHandlers();
openDocs.Add(modDoc, docHandler);
}
return true;
}

public bool DetachModelEventHandler(ModelDoc2 modDoc)
{
    DocumentEventHandler docHandler;
    docHandler = (DocumentEventHandler)openDocs[modDoc];
    openDocs.Remove(modDoc);
    modDoc = null;
    docHandler = null;
    return true;
}

public bool DetachEventHandlers()
{
    DetachSwEvents();

    // Close events on all currently open documents
    DocumentEventHandler docHandler;
    int numKeys = openDocs.Count;
    object[] keys = new Object[numKeys];

    // Remove all document event handlers
    openDocs.Keys.CopyTo(keys, 0);
    foreach (ModelDoc2 key in keys)
    {
        docHandler = (DocumentEventHandler)openDocs[key];
        //Remove the pair from the hash
        docHandler.DetachEventHandlers();
        docHandler = null;
    }
    return true;
}
#endregion

#region Event Handlers
// Events
public int ActiveDocChangeNotify()
{
    return 0;
}

public int DocumentLoadNotify2(string docTitle, string docPath)
{
    return 0;
}

public int FileOpenPostNotify(string FileName)
{
    AttachEventsToAllDocuments();
    return 0;
}

public int FileNewNotify2(object newDoc, int docType, string templateName)
{
    AttachEventsToAllDocuments();
    return 0;
}

public int ActiveModelDocChangeNotify()
{
    return 0;
}

#endregion
#region " UI Callbacks"
public void CScallbackFunction()
{
    resultCode = SwApp.ShowThirdPartyPopupMenu(registerID, 500, 500);
}

public int CSEnable()
{
    int functionReturnValue = 0;
    if (iSwApp.ActiveDoc == null)
    {
        functionReturnValue = 0;
    }
    else
    {
        functionReturnValue = 1;
    }
    return functionReturnValue;
}

#endregion
}
}

```

Add Button to Context-sensitive Menu Example (VB.NET)

This example shows how to add:

- button to a face's context-sensitive menu in a part.
- shortcut menu for that button.

```

-----
Preconditions:
1. Verify that the SOLIDWORKS API SDK add-in templates for your version
  of Microsoft Visual Studio are installed.
2. In Microsoft Visual Studio, create a Visual Basic project using the
  SwVBAddin template.
3. Name the project AddMenuPopupIconVBNET.
4. Copy and paste SwAddin into SwAddin.vb of your project.
5. Click Project > AddMenuPopupIconVBNET Properties > Debug.
6. Select Start external program and type the pathname to your
  SOLIDWORKS Design executable.
7. Replace the text shown for the imageList array elements
  and smallMainIcon with the pathnames to your image files.
8. Press F5 to start debugging this add-in.

Postconditions:
1. In SOLIDWORKS Design, open a part.
2. Right-click a face of the part.
3. Note the size of the button added by this add-in to the
  context-sensitive menu.
4. Click the button added by this add-in to display the shortcut menu.
5. Exit SOLIDWORKS Design.
6. Change the size of the text and other items on your computer.
  For example, in Windows 7:
  a. Click Start > Control Panel > Appearance and
    Personalization > Display.
  b. Select a different size.
  c. Click Apply.
  d. Click Log off now.
  e. Log back in.
7. Start SOLIDWORKS Design.
8. Repeat steps 1 - 3.
-----

'SwAddin
Imports System
Imports System.Collections
Imports System.Reflection
Imports System.Runtime.InteropServices

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swpublished
Imports SolidWorksTools
Imports SolidWorksTools.File

Imports System.Collections.Generic
Imports System.Diagnostics

<Guid("7af58543-3a92-417a-b682-9a3f20465cce")> _
<ComVisible(True)> _
<SwAddin(
  Description:="AddMenuPopupIconVBNET description", _
  Title:="AddMenuPopupIconVBNET", _
  LoadAtStartup:=True _
)>
  Public Class SwAddin
    Implements SolidWorks.Interop.swpublished.SwAddin

#Region "Local Variables"
    Dim WithEvents iSwApp As SldWorks
    Dim addinID As Integer
    Dim openDocs As Hashtable
    Dim SwEventPtr As SldWorks
    Dim iBmp As BitmapHandler
    Dim model As ModelDoc2
    Dim frame As IFrame
    Dim resultCode As Boolean
    Dim registerID As Integer

    ' Public Properties
    ReadOnly Property SwApp() As SldWorks
      Get
        Return iSwApp
      End Get
    End Property

    ReadOnly Property OpenDocumentsTable() As Hashtable
      Get
        Return openDocs
      End Get
    End Property
#End Region

#Region "SolidWorks Registration"

    <ComRegisterFunction()> Public Shared Sub RegisterFunction(ByVal t As Type)
      ' Get Custom Attribute: SwAddinAttribute
      Dim attributes() As Object
      Dim SwAttr As SwAddinAttribute = Nothing

      attributes = System.Attribute.GetCustomAttributes(GetType(SwAddin), GetType(SwAddinAttribute))

      If attributes.Length > 0 Then
        SwAttr = DirectCast(attributes(0), SwAddinAttribute)
      End If
      Try
        Dim hklm As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.LocalMachine
        Dim hkcu As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.CurrentUser

        Dim keyname As String = "SOFTWARE\SolidWorks\Addins\{" + t.GUID.ToString() + "}"
        Dim addinkey As Microsoft.Win32.RegistryKey = hklm.CreateSubKey(keyname)

```

```

        addinkey.SetValue(Nothing, 0)
        addinkey.SetValue("Description", Swattr.Description)
        addinkey.SetValue("Title", Swattr.Title)

        keyname = "Software\SolidWorks\AddInsStartup\" + t.GUID.ToString() + "}"
        addinkey = hkcu.CreateSubKey(keyname)
        addinkey.SetValue(Nothing, Swattr.LoadAtStartup, Microsoft.Win32.RegistryValueKind.DWord)
    Catch nl As System.NullReferenceException
        Console.WriteLine("There was a problem registering this dll: Swattr is null.\n " & nl.Message)
        System.Windows.Forms.MessageBox.Show("There was a problem registering this dll: Swattr is null.\n" & nl.Message)
    Catch e As System.Exception
        Console.WriteLine("There was a problem registering this dll: " & e.Message)
        System.Windows.Forms.MessageBox.Show("There was a problem registering this dll: " & e.Message)
    End Try
End Sub

<ComUnregisterFunction()> Public Shared Sub UnregisterFunction(ByVal t As Type)
    Try
        Dim hkml As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.LocalMachine
        Dim hkcu As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.CurrentUser

        Dim keyname As String = "SOFTWARE\SolidWorks\Addins\" + t.GUID.ToString() + "}"
        hkml.DeleteSubKey(keyname)

        keyname = "Software\SolidWorks\AddInsStartup\" + t.GUID.ToString() + "}"
        hkcu.DeleteSubKey(keyname)
    Catch nl As System.NullReferenceException
        Console.WriteLine("There was a problem unregistering this dll: Swattr is null.\n " & nl.Message)
        System.Windows.Forms.MessageBox.Show("There was a problem unregistering this dll: Swattr is null.\n" & nl.Message)
    Catch e As System.Exception
        Console.WriteLine("There was a problem unregistering this dll: " & e.Message)
        System.Windows.Forms.MessageBox.Show("There was a problem unregistering this dll: " & e.Message)
    End Try
End Sub

#End Region

#Region "ISwAddin Implementation"

Function ConnectToSW(ByVal ThisSW As Object, ByVal Cookie As Integer) As Boolean Implements SolidWorks.Interop.swpublished.SwAddin.ConnectToSW
    iSWApp = ThisSW
    addinID = Cookie

    ' Set up callbacks
    iSWApp.SetAddinCallbackInfo2(0, Me, addinID)

    ' Set up the third-party buttons
    AddThirdPartyIcon()

    ' Set up the Event Handlers
    SwEventPtr = iSWApp
    openDocs = New Hashtable
    AttachEventHandlers()

    ConnectToSW = True
End Function

Function DisconnectFromSW() As Boolean Implements SolidWorks.Interop.swpublished.SwAddin.DisconnectFromSW
    DetachEventHandlers()

    System.Runtime.InteropServices.Marshal.ReleaseComObject(iSWApp)
    iSWApp = Nothing
    ' The add-in must call GC.Collect() here to retrieve all managed code pointers
    GC.Collect()
    GC.WaitForPendingFinalizers()

    GC.Collect()
    GC.WaitForPendingFinalizers()

    DisconnectFromSW = True
End Function
#End Region

#Region "UI Methods"
Public Sub AddThirdPartyIcon()

    Dim iBmp As New BitmapHandler
    Dim thisAssembly As Assembly

    ' Create a button in the context-sensitive menus of faces in parts
    frame = SwApp.Frame()
    Dim imageList() As String = New String(2) {}
    imageList(0) = "Pathname_to_button_nxn_image"
    imageList(1) = "Pathname_to_button_nxxnn_image"
    imageList(2) = "Pathname_to_button_nnnxnnn_image"
    resultCode = frame.AddMenuPopupIcon3(swDocumentTypes_e.swDocPART, swSelectType_e.swSelfFACES, "Third-party context-sensitive", addinID, "CSCallbackFunction", "CSEnable", "", imageList)

    ' Create and register the shortcut menu
    registerID = SwApp.RegisterThirdPartyPopupMenu()
    ' Add a menu break at the top of the shortcut menu
    resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, CInt(swDocumentTypes_e.swDocPART), "Menu Break", addinID, "", "", "", "", "", CInt(swMenuItemType_e.swMenuItemType_Break))
    ' Add a couple of items to the shortcut menu
    Dim smallMainIcon As String
    smallMainIcon = "Pathname_to_shortcut_button_image"
    resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, CInt(swDocumentTypes_e.swDocPART), "Test1", addinID, "TestCallback", "EnableTest", "", , "Test1", smallMainIcon, CInt(swMenuItemType_e.swMenuItemType_Default))
    resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, CInt(swDocumentTypes_e.swDocPART), "Test4", addinID, "TestCallback", "EnableTest", "", , "Test4", smallMainIcon, CInt(swMenuItemType_e.swMenuItemType_Default))
    ' Add a separator bar to the shortcut menu
    resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, CInt(swDocumentTypes_e.swDocPART), "separator", addinID, "", "", "", "", "", CInt(swMenuItemType_e.swMenuItemType_Separator))
    ' Add another item to the shortcut menu
    resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, CInt(swDocumentTypes_e.swDocPART), "Test5", addinID, "TestCallback", "EnableTest", "", , "Test5", smallMainIcon, CInt(swMenuItemType_e.swMenuItemType_Default))
    ' Add a button to a menu bar of the shortcut menu
    resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, CInt(swDocumentTypes_e.swDocPART), "", addinID, "TestCallback", "EnableTest", "", "NoOp", smallMainIcon, CInt(swMenuItemType_e.swMenuItemType_Default))

    thisAssembly = Nothing
    iBmp.Dispose()
End Sub

Function CompareIDs(ByVal storedIDs() As Integer, ByVal addinIDs() As Integer) As Boolean

```

```

Dim storeList As New List(Of Integer)(storedIDs)
Dim addinList As New List(Of Integer)(addinIDs)

addinList.Sort()
storeList.Sort()

If Not addinList.Count = storeList.Count Then
    Return False
Else
    For i As Integer = 0 To addinList.Count - 1
        If Not addinList(i) = storeList(i) Then
            Return False
        End If
    Next
End If

Return True
End Function
#End Region

#Region "Event Methods"

Sub AttachEventHandlers()
    AttachSWEvents()
    ' Listen for events on all currently open documents
    AttachEventsToAllDocuments()
End Sub

Sub DetachEventHandlers()
    DetachSWEvents()

    ' Close events on all currently open documents
    Dim docHandler As DocumentEventHandler
    Dim key As ModelDoc2
    Dim numKeys As Integer
    numKeys = openDocs.Count
    If numKeys > 0 Then
        Dim keys() As Object = New Object(numKeys - 1) {}

        ' Remove all document event handlers
        openDocs.Keys.CopyTo(keys, 0)
        For Each key In keys
            docHandler = openDocs.Item(key)
            ' Remove the pair from the hash
            docHandler.DetachEventHandlers()
            docHandler = Nothing
            key = Nothing
        Next
    End If
End Sub

Sub AttachSWEvents()
    Try
        AddHandler iSwApp.ActiveDocChangeNotify, AddressOf Me.SldWorks_ActiveDocChangeNotify
        AddHandler iSwApp.DocumentLoadNotify2, AddressOf Me.SldWorks_DocumentLoadNotify2
        AddHandler iSwApp.FileNewNotify2, AddressOf Me.SldWorks_FileNewNotify2
        AddHandler iSwApp.ActiveModelDocChangeNotify, AddressOf Me.SldWorks_ActiveModelDocChangeNotify
        AddHandler iSwApp.FileOpenPostNotify, AddressOf Me.SldWorks_FileOpenPostNotify
    Catch e As Exception
        Console.WriteLine(e.Message)
    End Try
End Sub

Sub DetachSWEvents()
    Try
        RemoveHandler iSwApp.ActiveDocChangeNotify, AddressOf Me.SldWorks_ActiveDocChangeNotify
        RemoveHandler iSwApp.DocumentLoadNotify2, AddressOf Me.SldWorks_DocumentLoadNotify2
        RemoveHandler iSwApp.FileNewNotify2, AddressOf Me.SldWorks_FileNewNotify2
        RemoveHandler iSwApp.ActiveModelDocChangeNotify, AddressOf Me.SldWorks_ActiveModelDocChangeNotify
        RemoveHandler iSwApp.FileOpenPostNotify, AddressOf Me.SldWorks_FileOpenPostNotify
    Catch e As Exception
        Console.WriteLine(e.Message)
    End Try
End Sub

Sub AttachEventsToAllDocuments()
    Dim modDoc As ModelDoc2
    modDoc = iSwApp.GetFirstDocument()
    While Not modDoc Is Nothing
        If Not openDocs.Contains(modDoc) Then
            AttachModelDocEventHandler(modDoc)
        Else
            Dim docHandler As DocumentEventHandler = openDocs(modDoc)
            If Not docHandler Is Nothing Then
                docHandler.ConnectModelViews()
            End If
        End If
        modDoc = modDoc.GetNext()
    End While
End Sub

Function AttachModelDocEventHandler(ByVal modDoc As ModelDoc2) As Boolean
    If modDoc Is Nothing Then
        Return False
    End If
    Dim docHandler As DocumentEventHandler = Nothing

    If Not openDocs.Contains(modDoc) Then
        Select Case modDoc.GetType
            Case swDocumentTypes_e.swDocPART
                docHandler = New PartEventHandler()
            Case swDocumentTypes_e.swDocASSEMBLY
                docHandler = New AssemblyEventHandler()
            Case swDocumentTypes_e.swDocDRAWING
                docHandler = New DrawingEventHandler()
        End Select

        docHandler.Init(iSwApp, Me, modDoc)
        docHandler.AttachEventHandlers()
        openDocs.Add(modDoc, docHandler)
    End If
End Function

Sub DetachModelDocEventHandler(ByVal modDoc As ModelDoc2)
    Dim docHandler As DocumentEventHandler
    docHandler = openDocs.Item(modDoc)
    openDocs.Remove(modDoc)

```

```

        modDoc = Nothing
        docHandler = Nothing
    End Sub
#End Region

#Region "Event Handlers"
Function SldWorks_ActiveDocChangeNotify() As Integer
    ' TODO: Add your implementation here
End Function

Function SldWorks_DocumentLoadNotify2(ByVal docTitle As String, ByVal docPath As String) As Integer
    ' TODO: Add your implementation here
End Function

Function SldWorks_FileNewNotify2(ByVal newDoc As Object, ByVal doctype As Integer, ByVal templateName As String) As Integer
    AttachEventsToAllDocuments()
End Function

Function SldWorks_ActiveModelDocChangeNotify() As Integer
    ' TODO: Add your implementation here
End Function

Function SldWorks_FileOpenPostNotify(ByVal FileName As String) As Integer
    AttachEventsToAllDocuments()
End Function
#End Region

#Region " UI Callbacks"
Sub CS_CALLBACK_Function()
    resultCode = SwApp.ShowThirdPartyPopupMenu(registerID, 500, 500)
End Sub

Function CSEnable() As Integer
    If iSwApp.ActiveDoc Is Nothing Then
        CSEnable = 0
    Else
        CSEnable = 1
    End If
End Function
#End Region

End Class

```

Add Buttons to Task Pane Example (C#)

This example shows how to add standard SOLIDWORKS and custom buttons to the Task Pane.

```
//-----
// 1. Copy save.png from public_documents\samples\tutorial\api to this macro's
// SwMacro folder.
// 2. Clear the Stop VSTA debugger on macro exit check box if selected
// in Tools > Options > System Options.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Opens the Task Pane and loads the My Calendar control with
// the specified standard and custom buttons at the
// top of the pane.
// 2. Click each button from left to right and examine the Immediate
// window after each click.
// 3. Click the Stop Debugging button in the IDE.
// 4. Select the Stop VSTA debugger on macro exit check box in
// Tools > Options > System Options if cleared in
// Preconditions step 2.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Collections;
using System.Windows.Forms;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public TaskpaneView swTaskPane;
        public int buttonIdx;

        public void Main()
        {
            bool result = false;
            string folder = null;
            string bitmap = null;
            string toolTip = null;
            string ctrlName = null;
            string ctrlLicKey = null;

            folder = swApp.GetCurrentMacroPathFolder() + "\\";

            // Use default image for Task Pane tab
            bitmap = "";
            toolTip = "My Calendar";
            ctrlName = "My.Calendar";
            ctrlLicKey = "";
            swTaskPane = (TaskpaneView)swApp.CreateTaskpaneView2(bitmap, toolTip);

            // Add standard and custom buttons to Task Pane
            result = swTaskPane.AddCustomButton(folder + "save.png", "Save (custom png)");
            result = swTaskPane.AddStandardButton((int)swTaskPaneBitmapsOptions_e.swTaskPaneBitmapsOptions_Next, "Next (standard)");
            result = swTaskPane.AddStandardButton((int)swTaskPaneBitmapsOptions_e.swTaskPaneBitmapsOptions_Back, "Back (standard)");
            result = swTaskPane.AddStandardButton((int)swTaskPaneBitmapsOptions_e.swTaskPaneBitmapsOptions_Ok, "OK (standard)");
            result = swTaskPane.AddStandardButton((int)swTaskPaneBitmapsOptions_e.swTaskPaneBitmapsOptions_Close, "Close (standard)");

            // Add control to Task Pane for the buttons
            swTaskPane.AddControl(ctrlName, ctrlLicKey);

            // Set up events

            AttachEventHandlers();
        }

        public void AttachEventHandlers()
        {
            AttachSWEvents();
        }

        public void AttachSWEvents()
        {
            swTaskPane.TaskPaneActivateNotify += this.swTaskPane_TaskPaneActivateNotify;
            swTaskPane.TaskPaneDestroyNotify += this.swTaskPane_TaskPaneDestroyNotify;
            swTaskPane.TaskPaneToolBarButtonClicked += this.swTaskPane_TaskPaneToolBarButtonClicked;
        }

        public int swTaskPane_TaskPaneActivateNotify()
        {
            if (swTaskPane.GetButtonState(0) == false)
            {
                for (buttonIdx = 0; buttonIdx <= 20; buttonIdx++)
                {

```

```

        swTaskPane.SetButtonState(buttonIdx, true);
    }
}
else
{
    for (buttonIdx = 0; buttonIdx <= 20; buttonIdx++)
    {
        swTaskPane.SetButtonState(buttonIdx, false);
    }
}
return 0;
}

public int swTaskPane_TaskPaneDestroyNotify()
{
    MessageBox.Show("Remove control from Task Pane?");
    return 1;
}

public int swTaskPane_TaskPaneToolbarButtonClicked(int ButtonIndex)
{
    switch ((ButtonIndex + 1))
    {
        case 1:
            Debug.Print("Save (custom png) button clicked.");
            break;
        case 2:
            Debug.Print("Next button clicked.");
            break;
        case 3:
            Debug.Print("Back button clicked.");
            break;
        case 4:
            Debug.Print("Okay button clicked.");
            break;
        case 5:
            Debug.Print("Close button clicked and tab deleted.");
            swTaskPane.DeleteView();
            break;
    }
    return 1;
}
}

```

```

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;

```

```

    }
}

```

Add Buttons to Task Pane Example (VBA)

This example shows how to add standard SOLIDWORKS and custom buttons to the Task Pane.

```
'-----
' Preconditions:
' 1. Copy Main to the main module.
' 2. Click Insert > Class Module and copy Class to that module.
' 3. Copy save.png from public_documents\samples\tutorial\api to
'    this macro 's folder.
'
' Postconditions:
' 1. Opens the Task Pane and loads the My Calendar control with
'    the specified standard and custom buttons at the
'    top of the pane.
' 2. Click each button from left to right.
' 3. Displays a message box confirming each button click.
' 4. Click OK to close each message box.
'    a. Clicking the Close button prompts you to remove the
'       control from the Task Pane.
'    b. Click OK.
'-----
'Main
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swTaskPane As SldWorks.TaskpaneView
Dim result As Boolean
Dim folder As String
Dim bitmap As String
Dim toolTip As String
Dim ctrlName As String
Dim ctrlLicKey As String
Public buttonIdx As Integer
Dim swTaskpaneEvents As New Class1

Sub main()

    Set swApp = Application.SldWorks
    folder = swApp.GetCurrentMacroPathFolder + "\"

    ' Use default image for Task Pane tab
    bitmap = ""
    toolTip = "My Calendar"
    ctrlName = "My.Calendar"
    ctrlLicKey = ""
    Set swTaskPane = swApp.CreateTaskpaneView2(bitmap, toolTip)

    ' Add standard and custom buttons to Task Pane
    result = swTaskPane.AddCustomButton(folder & "save.png", "Save (custom png)")
    result = swTaskPane.AddStandardButton(swTaskPane.BitmapsOptions_Next, "Next (standard)")
    result = swTaskPane.AddStandardButton(swTaskPane.BitmapsOptions_Back, "Back (standard)")
    result = swTaskPane.AddStandardButton(swTaskPane.BitmapsOptions_Ok, "OK (standard)")
    result = swTaskPane.AddStandardButton(swTaskPane.BitmapsOptions_Close, "Close (standard)")

    ' Add control to Task Pane for the buttons
    swTaskPane.AddControl ctrlName, ctrlLicKey

    ' Set up events
    Set swTaskpaneEvents.swTaskPane = swTaskPane
```

End Sub

[Back to top](#)

'Class

Option Explicit

Public WithEvents swTaskPane As SldWorks.TaskpaneView

```
Private Function swTaskPane_TaskPaneActivateNotify() As Long
    If (swTaskPane.GetButtonState(0) = False) Then
        For buttonIdx = 0 To 20
            swTaskPane.SetButtonState buttonIdx, True
        Next
    Else
        For buttonIdx = 0 To 20
            swTaskPane.SetButtonState buttonIdx, False
        Next
    End If
End Function
```

```
Private Function swTaskPane_TaskPaneDestroyNotify() As Long
    MsgBox ("Remove control from Task Pane?")
End Function
```

```
Private Function swTaskPane_TaskPaneToolBarButtonClicked(ByVal ButtonIndex As Long) As Long
    Select Case (ButtonIndex + 1)
        Case 1
            MsgBox "Save (custom png) button clicked."
        Case 2
            MsgBox "Next button clicked."
        Case 3
```

```
        MsgBox "Back button clicked."
    Case 4
        MsgBox "Okay button clicked."
    Case 5
        MsgBox "Close button clicked."
        swTaskPane.DeleteView
    End Select
End Function
```

[Back to top](#)

Add Buttons to Task Pane Example (VB.NET)

This example shows how to add standard SOLIDWORKS and custom buttons to the Task Pane.

```
'-----
' 1. Copy save.png from public_documents\samples\tutorial\api to this macro's
'   SwMacro folder.
' 2. Clear the Stop VSTA debugger on macro exit check box if selected
'   in Tools > Options > System Options.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Opens the Task Pane and loads the My Calendar control with
'   the specified standard and custom buttons at the
'   top of the pane.
' 2. Click each button from left to right and examine the Immediate
'   window after each click.
' 3. Click the Stop Debugging button in the IDE.
' 4. Select the Stop VSTA debugger on macro exit check box in
'   Tools > Options > System Options if cleared in
'   Preconditions step 2.
'-----
```

Imports SolidWorks.Interop.sldworks

Imports SolidWorks.Interop.swconst

Imports System

Imports System.Collections

Imports System.Diagnostics

Partial Class SolidWorksMacro

Public WithEvents swTaskPane As TaskpaneView

Public buttonIdx As Integer

Public Sub Main()

Dim result As Boolean

Dim folder As String

Dim bitmap As String

Dim toolTip As String

Dim ctrlName As String

Dim ctrlLicKey As String

folder = swApp.GetCurrentMacroPathFolder + "\"

' Use default
image for Task Pane tab

bitmap = ""

toolTip = "My Calendar"

ctrlName = "My.Calendar"

ctrlLicKey = ""

swTaskPane = swApp.CreateTaskpaneView2(bitmap, toolTip)

' Add standard and custom buttons to Task
Pane

result = swTaskPane.AddCustomButton(folder & "save.png", "Save (custom png)")

result = swTaskPane.AddStandardButton(swTaskPaneBitmapsOptions_e.swTaskPaneBitmapsOptions_Next, "Next (standard)")

result = swTaskPane.AddStandardButton(swTaskPaneBitmapsOptions_e.swTaskPaneBitmapsOptions_Back, "Back (standard)")

result = swTaskPane.AddStandardButton(swTaskPaneBitmapsOptions_e.swTaskPaneBitmapsOptions_Ok, "OK (standard)")

result = swTaskPane.AddStandardButton(swTaskPaneBitmapsOptions_e.swTaskPaneBitmapsOptions_Close, "Close (standard)")

' Add control to Task Pane for the buttons

```

swTaskPane.AddControl(ctrlName, ctrlLicKey)

' Set up events
AttachEventHandlers()

End Sub

Sub AttachEventHandlers()
    AttachSWEvents()
End Sub

Sub AttachSWEvents()
    AddHandler swTaskPane.TaskPaneActivateNotify, AddressOf Me.swTaskPane_TaskPaneActivateNotify
    AddHandler swTaskPane.TaskPaneDestroyNotify, AddressOf Me.swTaskPane_TaskPaneDestroyNotify
    AddHandler swTaskPane.TaskPaneToolBarButtonClicked, AddressOf Me.swTaskPane_TaskPaneToolBarButtonClicked
End Sub

Public Function swTaskPane_TaskPaneActivateNotify() As Integer
    If Not
        (swTaskPane.GetButtonState(0)) Then
        For buttonIdx = 0 To 20
            swTaskPane.SetButtonState(buttonIdx,
True)

            Next
        Else
            For buttonIdx = 0 To 20
                swTaskPane.SetButtonState(buttonIdx,
False)

            Next
        End If
    End Function

Public Function swTaskPane_TaskPaneDestroyNotify() As Integer
    MsgBox("Remove control from Task Pane?")
End Function

Public Function swTaskPane_TaskPaneToolBarButtonClicked(ByVal ButtonIndex As Integer) As Integer
    Select Case (ButtonIndex + 1)
        Case 1
            Debug.Print("Save (custom png) button clicked.")
        Case 2
            Debug.Print("Next button clicked.")
        Case 3
            Debug.Print("Back button clicked.")
        Case 4
            Debug.Print("Okay button clicked.")
        Case 5
            Debug.Print("Close button clicked and tab deleted.")
            swTaskPane.DeleteView()
    End Select

End Function

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>

Public swApp As SldWorks

```

End Class

Add Comment to Assembly Component Example (C#)

This example shows how to add a comment to an assembly component.

```
//-----
// Preconditions:
// 1. Open an assembly document.
// 2. Select a component in the FeatureManager design tree.
// 3. Open an Immediate window.
//
// Postconditions: A comment about the selected component is added to the
// document's Comments folder.
// -----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace AddComment_C_.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel;
            SelectionMgr swSelMgr;
            ModelDocExtension swModelDocExt;
            Component2 selComp;
            String selCompName;
            Comment newComment;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swModelDocExt = swModel.Extension;

            selComp = (Component2)swSelMgr.GetSelectedObjectsComponent4(1, -1);
            selCompName = selComp.Name2;

            newComment = swModelDocExt.AddComment("This component's name is " + selCompName);
        }
    }
    public SldWorks swApp;
}
```

Add Comment to Assembly Component Example (VBA)

This example shows how to add a comment to an assembly component.

```
'-----  
' Preconditions:  
' 1. Open an assembly document.  
' 2. Select a component in the FeatureManager design tree.  
' 3. Open an Immediate window.  
'  
' Postconditions: A comment about the selected component is added to the  
' document's Comments folder.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swModelDocExt As SldWorks.ModelDocExtension
```

Sub main()

```
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swModelDocExt = swModel.Extension
```

```
    Dim selComp As SldWorks.Component2  
    Dim selCompName As String  
    Set selComp = swSelMgr.GetSelectedObjectsComponent4(1, -1)
```

```
    selCompName = selComp.Name2
```

```
    Dim newComment As SldWorks.Comment  
    Set newComment = swModelDocExt.AddComment("This component's name is " & selCompName)
```

End Sub

Add Comment to Assembly Component Example (VB.NET)

This example shows how to add a comment to an assembly component.

```
'-----  
' Preconditions:  
' 1. Open an assembly document.  
' 2. Select a component in the FeatureManager design tree.  
' 3. Open an Immediate window.  
'-----  
' Postconditions: A comment about the selected component is added to the  
' document's Comments folder.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub main()
```

```
        Dim swModel As ModelDoc2  
        Dim swSelMgr As SelectionMgr  
        Dim swModelDocExt As ModelDocExtension  
        Dim selComp As Component2  
        Dim selCompName As String  
        Dim newComment As Comment
```

```
        swModel = swApp.ActiveDoc  
        swSelMgr = swModel.SelectionManager  
        swModelDocExt = swModel.Extension
```

```
        selComp = swSelMgr.GetSelectedObjectsComponent4(1, -1)  
        selCompName = selComp.Name2
```

```
        newComment = swModelDocExt.AddComment("This component's name is " & selCompName)
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Add Component and Mate Example (C++ COM)

This example shows how to add a component and mate to an assembly.

- [AddMate.cpp](#)
- [stdafx.h](#)
- [stdafx.cpp](#)

AddMate.cpp

```
////////////////////////////////////
//
// Preconditions: The specified assembly (lens_mount.sldasm) and component
//               (camtest.sldprt) files exist.
//
// NOTE: The assembly and component files are installed when you
//         install SOLIDWORKS Design and are intended for the SOLIDWORKS Toolbox
//         tutorial, so do not save the assembly after running the program.
//
// Postconditions: The component and a mate, called top_coinc_camtest-1,
//                are added to the assembly.
//
////////////////////////////////////

#include "stdafx.h"

using namespace std;

int _tmain(int argc, _TCHAR* argv[])
{
    //Initialize COM
    CoInitialize(NULL);

    //Use ATL smart pointers
    CComPtr<ISldWorks> swApp;
    CComPtr<IModelDoc2> swModel;
    CComPtr<IModelDocExtension> swDocExt;
    CComPtr<IAssemblyDoc> swAssy;
    CComPtr<IComponent2> swComponent;
    CComPtr<IFeature> mateFeature;
    CComPtr<IModelDoc2> tmpObj;
    CComPtr<IMate2> swMate;

    long lErrors;
    long lWarnings;
    long lMateError;

    HRESULT hres = NOERROR;
    VARIANT_BOOL retVal = VARIANT_FALSE;

    //Connect to currently running instance of SOLIDWORKS Design
    hres = swApp.CoCreateInstance(__uuidof(SldWorks), NULL, CLSCTX_LOCAL_SERVER);

    //Open the assembly document
    COMBSTR sAssemblyName(L"C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\toolbox\\lens_mount.sldasm");
    COMBSTR sDefaultConfiguration(L"Default");
    swApp->OpenDoc6(sAssemblyName, swDocASSEMBLY, swOpenDocOptions_Silent, sDefaultConfiguration, &lErrors, &lWarnings, &swModel);

    swModel = NULL;

    //Open the component (part) document
    COMBSTR sCompName(L"C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\toolbox\\camtest.sldprt");
    swApp->OpenDoc6(sCompName, swDocPART, swOpenDocOptions_Silent, sDefaultConfiguration, &lErrors, &lWarnings, &tmpObj);

    //Re-activate the assembly document
    swApp->IActivateDoc3(sAssemblyName, true, &lErrors, &swModel);

    swAssy = swModel;
```

```

//Add the component to the assembly
swAssy->AddComponent4(sCompName, sDefaultConfiguration, -1, -1, -1, &swComponent);

//Concatenate strings for name of mate
CComBSTR sMateName(L"top_coinc_");
CComBSTR sCompMateName (L"");
swComponent->get_Name2(&sCompMateName);
sMateName.AppendBSTR(sCompMateName);
CComBSTR sFullCompMateName = sMateName;

//Concatenate strings for name of first selection: Top plane of component
CComBSTR sFirstSelection(L"Top@");
sFirstSelection.AppendBSTR(sCompMateName);
CComBSTR sAtSign(L"@");
sFirstSelection.AppendBSTR(sAtSign);
CComBSTR sAssemblyFilename(L"lens_mount");
sFirstSelection.AppendBSTR(sAssemblyFilename);

//Second selection: Front plane of assembly
CComBSTR sSecondSelection(L"Front@");

//Adjust the view so that you can see the assembly and the added component
CComBSTR sNamedView(L"*Trimetric");
swModel->ShowNamedView2(sNamedView, -1);
swModel->ViewZoomtofit2();

swModel->ClearSelection2(true);

swModel->get_Extension(&swDocExt);

//Select the planes to mate
CComBSTR sPlane(L"PLANE");
swDocExt->SelectByID2(sFirstSelection, sPlane, 0, 0, 0, true, 1, NULL, swSelectOptionDefault, &retVal);
swDocExt->SelectByID2(sSecondSelection, sPlane, 0, 0, 0, true, 1, NULL, swSelectOptionDefault, &retVal);

//Mate the selected entities
swAssy->AddMate3(swMateCOINCIDENT, swMateAlignALIGNED, false, 0, 0, 0, 0, 0, 0, false, &IMateError, &swMate);

//Set the name of the mate feature
mateFeature = swMate;
mateFeature->put_Name(sFullCompMateName);

return 0;
}

```

[Back to top](#)

stdafx.h

```

// stdafx.h : include file for standard system include files,
// or project-specific include files that are used frequently, but
// are changed infrequently
//

#pragma once

#define WIN32_LEAN_AND_MEAN // Exclude rarely-used stuff from Windows headers
#include <stdio.h>
#include <tchar.h>
#define _ATL_CSTRING_EXPLICIT_CONSTRUCTORS // some CString constructors will be explicit

#include <windows.h>
#include <atbase.h>
#include <iostream>

using namespace std; //Use the standard C++ libraries for text output.
#import "sldworks.tlb" raw_interfaces_only, raw_native_types, no_namespace, named_guids //the SOLIDWORKS type library
#import "swconst.tlb" raw_interfaces_only, raw_native_types, no_namespace, named_guids //the SOLIDWORKS constant type library

// TODO: reference additional headers your program requires here

```

[Back to top](#)

stdafx.cpp

```
// stdafx.cpp : source file that includes just the standard includes
// AddMates.pch will be the pre-compiled header
// stdafx.obj will contain the pre-compiled type information

#include "stdafx.h"

// TODO: reference any additional headers you need in STDAFX.H
// and not in this file
```

[Back to top](#)

Add and Mate Component Example (C#)

This example shows how to add a component to an assembly and mate it.

```
//-----
// Preconditions:
// 1. Verify that these documents exist in public_documents\samples\tutorial\toolbox:
//    * lens_mount.sldasm
//    * camtest.sldprt
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens lens_mount.sldasm.
// 2. Adds the specified component, camtest.sldprt, to the assembly.
// 3. Fires the AddItemNotify event.
// 4. Makes the specified component virtual by saving it to the
//    assembly with a new name.
// 5. Fires the RenameItemNotify event.
// 6. Adds a mate between the selected planes to the assembly.
// 7. Inspect the Immediate window and FeatureManager design tree.
//
// NOTE: Because the models are used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;
using System.Collections;
using System.Windows.Forms;

namespace MakeVirtualCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public AssemblyDoc swAssemblyDoc;
        ModelDoc2 swModel;
        ModelDocExtension swDocExt;
        Hashtable openAssem;
        string tmpPath;
        ModelDoc2 tmpObj;
        bool boolstat;
        bool stat;
        Component2 swComponent;
        Feature mateFeature;
        string MateName;
        string FirstSelection;
        string SecondSelection;
        string strCompName;
        string AssemblyTitle;
        string AssemblyName;
        int errors;
        int warnings;
        string fileName;

        int mateError;

        public void Main()
        {
            // Open assembly
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\toolbox\\lens_mount.sldasm";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY,
            (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);

            // Set up event
            swAssemblyDoc = (AssemblyDoc)swModel;
            openAssem = new Hashtable();
            AttachEventHandlers();

            // Get title of assembly document
            AssemblyTitle = swModel.GetTitle();

            // Split the title into two strings using the period as the delimiter
            string[] strings = AssemblyTitle.Split(new Char[] { '.' });

            // Use AssemblyName when mating the component with the assembly
            AssemblyName = (string)strings[0];

            Debug.Print("Name of assembly: " + AssemblyName);

            boolstat = true;
            string strCompModelname = null;
            strCompModelname = "camtest.sldprt";

            // Because the component resides in the same folder as the assembly, get
```

```

// the assembly's path and use it when opening the component
tmpPath = swModel.GetPathName();
int idx;
idx = tmpPath.LastIndexOf("lens_mount.sldasm");
string compPath;
tmpPath = tmpPath.Substring(0, (idx));
compPath = string.Concat(tmpPath, strCompModelName);

// Open the component
tmpObj = (ModelDoc2)swApp.OpenDoc6(compPath, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Sil

// Check to see if the file is read-only or cannot be found; display error
// messages if either
if (warnings == (int)swFileLoadWarning_e.swFileLoadWarning_ReadOnly)
{
    MessageBox.Show("This file is read-only.");
    boolstat = false;
}

if (tmpObj == null)
{
    MessageBox.Show("Cannot locate the file.");
    boolstat = false;
}

// Activate the assembly so that you can add the component to it
swModel = (ModelDoc2)swApp.ActivateDoc3(AssemblyTitle, true, (int)swRebuildOnActivation_e.swUserDecision, ref errors);

// Add the part to the assembly document
swComponent = (Component2)swAssemblyDoc.AddComponent5(strCompModelName, (int)swAddComponentConfigOptions_e.swAddComponentCo

// Make the component virtual
stat = swComponent.MakeVirtual2(true);

// Get the name of the component for the mate
strCompName = swComponent.Name2;

// Create the name of the mate and the names of the planes to use for the mate
MateName = "top_coinc_" + strCompName;
FirstSelection = "Top@" + strCompName + "@" + AssemblyName;
SecondSelection = "Front@" + AssemblyName;

swModel.ClearSelection2(true);
swDocExt = (ModelDocExtension)swModel.Extension;

// Select the planes for the mate
boolstat = swDocExt.SelectByID2(FirstSelection, "PLANE", 0, 0, 0, true, 1, null, (int)swSelectOption_e.swSelectOptionDefaul
boolstat = swDocExt.SelectByID2(SecondSelection, "PLANE", 0, 0, 0, true, 1, null, (int)swSelectOption_e.swSelectOptionDefau

// Add the mate
matefeature = (Feature)swAssemblyDoc.AddMate5((int)swMateType_e.swMateCOINCIDENT, (int)swMateAlign_e.swMateAlignALIGNED, fa
0, out mateError);
matefeature.Name = MateName;
Debug.Print("Mate added: " + matefeature.Name);

swModel.ViewZoomtofit2();

}

public void AttachEventHandlers()
{
    AttachSWEvents();
}

public void AttachSWEvents()
{
    {
        swAssemblyDoc.AddItemNotify += this.swAssemblyDoc_AddItemNotify;
        swAssemblyDoc.RenameItemNotify += this.swAssemblyDoc_RenameItemNotify;
    }

private int swAssemblyDoc_AddItemNotify(int EntityType, string itemName)
{
    Debug.Print("Component added: " + itemName);
    return 1;
}

private int swAssemblyDoc_RenameItemNotify(int EntityType, string oldName, string NewName)
{
    Debug.Print("Virtual component name: " + NewName);
    return 1;
}

public SldWorks swApp;
}
}

```

Add and Mate Component Example (VBA)

This example shows how to add a component to an assembly and mate it.

```

'-----
' Preconditions:
' 1. Copy and paste this code in the main module.
' 2. Click Insert > Class module and copy and paste this code in the class
'    module.
' 3. Verify that these documents exist in public_documents\samples\tutorial\toolbox:
'    * lens_mount.sldasm
'    * camtest.sldprt
' 4. Open the Immediate window.
'
' Postconditions:
' 1. Opens lens_mount.sldasm.
' 2. Adds the specified component, camtest.sldprt, to the assembly.
' 3. Fires the AddItemNotify event.
' 4. Makes the specified component virtual by saving it to the
'    assembly with a new name.
' 5. Fires the RenameItemNotify event.
' 6. Adds a mate between the selected planes to the assembly.
' 7. Inspect the Immediate window and FeatureManager design tree.
'
' NOTE: Because the models are used elsewhere, do not save changes.
'-----

'Main module
Option Explicit

Dim swApp As New SldWorks.SldWorks
Dim swModel As ModelDoc2
Dim swDocExt As ModelDocExtension
Dim swAssy As AssemblyDoc
Dim swAssyEvents As Class1
Dim tmpPath As String
Dim tmpObj As SldWorks.ModelDoc2
Dim boolstat As Boolean, stat As Boolean
Dim strings As Variant
Dim swcomponent As SldWorks.Component2
Dim mateFeature As SldWorks.Feature
Dim MateName As String
Dim FirstSelection As String
Dim SecondSelection As String
Dim strCompName As String
Dim AssemblyTitle As String
Dim AssemblyName As String
Dim errors As Long
Dim warnings As Long
Dim mateError As Long
Dim fileName As String

Sub main()
    Set swApp = CreateObject("SldWorks.Application")

    ' Open assembly
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\toolbox\lens_mount.sldasm"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

    ' Get title of assembly document
    AssemblyTitle = swModel.GetTitle

    ' Split the title into two strings using the period as the delimiter
    strings = Split(AssemblyTitle, ".")

    ' Use AssemblyName when mating the component with the assembly
    AssemblyName = strings(0)

    Debug.Print "Name of assembly: " & AssemblyName

    boolstat = True
    Dim strCompModelname As String
    strCompModelname = "camtest.sldprt"

    ' Because the component resides in the same folder as the assembly, get
    ' the assembly's path and use it when opening the component
    tmpPath = Left(swModel.GetPathName, InStrRev(swModel.GetPathName, "\"))

    ' Open the component
    Set tmpObj = swApp.OpenDoc6(tmpPath + strCompModelname, swDocPART, 0, "", errors, warnings)

    ' Check to see if the file is read-only or cannot be found; display error messages
    If warnings = swFileLoadWarning_ReadOnly Then
        MsgBox "This file is read-only."
        boolstat = False
    End If

```

```

If tmpObj Is Nothing Then
    MsgBox "Cannot locate the file."
    boolstat = False
End If

' Activate the assembly so that you can add the component to it
Set swModel = swApp.ActivateDoc3(AssemblyTitle, True, swUserDecision, errors)

' Set up events
Set swAssy = swModel
Set swAssyEvents = New Class1
Set swAssyEvents.swAssy = swApp.ActiveDoc

' Add the part to the assembly document
Set swcomponent = swAssy.AddComponent5(strCompModelname, swAddComponentConfigOptions_CurrentSelectedConfig, "", False, "", -1,
-1, -1)

' Make the component virtual
stat = swcomponent.MakeVirtual2(True)

' Get the name of the component for the mate
strCompName = swcomponent.Name2()

' Create the name of the mate and the names of the planes to use for the mate
MateName = "top_coinc_" + strCompName
FirstSelection = "Top@" + strCompName & "@" + AssemblyName
SecondSelection = "Front@" + AssemblyName

swModel.ClearSelection2 (True)
Set swDocExt = swModel.Extension

' Select the planes to mate
boolstat = swDocExt.SelectByID2(FirstSelection, "PLANE", 0, 0, 0, True, 1, Nothing, swSelectOptionDefault)
boolstat = swDocExt.SelectByID2(SecondSelection, "PLANE", 0, 0, 0, True, 1, Nothing, swSelectOptionDefault)

' Add the mate
Set matefeature = swAssy.AddMate5(swMateCOINCIDENT, swMateAlignALIGNED, False, 0, 0, 0, 0, 0, 0, 0, 0, 0, False, False, 0,
mateError)
matefeature.Name = MateName
Debug.Print "Mate added: " & matefeature.Name

swModel.ViewZoomtofit2
End Sub

```

[Back to top](#)

'Class module

```

Option Explicit
Public WithEvents swAssy As SldWorks.AssemblyDoc

Private Function swAssy_AddItemNotify(ByVal EntityType As Long, ByVal itemName As String) As Long
    Debug.Print "Component added: " & itemName
End Function

Private Function swAssy_RenameItemNotify(ByVal EntityType As Long, ByVal oldName As String, ByVal NewName As String) As Long
    Debug.Print "Virtual component name: " & NewName
End Function

```

[Back to top](#)

Add and Mate Component Example (VB.NET)

This example shows how to add a component to an assembly and mate it.

```

'-----
' Preconditions:
' 1. Verify that these documents exist in public_documents\samples\tutorial\toolbox:
'   * lens_mount.sldasm
'   * camtest.sldprt
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens lens_mount.sldasm.
' 2. Adds the specified component, camtest.sldprt, to the assembly.
' 3. Fires the AddItemNotify event.
' 4. Makes the specified component virtual by saving it to the
'   assembly with a new name.
' 5. Fires the RenameItemNotify event.
' 6. Adds a mate between the selected planes to the assembly.
' 7. Inspect the Immediate window and FeatureManager design tree.
'
' NOTE: Because the models are used elsewhere, do not save changes.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics
Imports System.Collections

```

```
Partial Class SolidWorksMacro
```

```
    Public WithEvents swAssemblyDoc As AssemblyDoc
```

```

    Dim swModel As ModelDoc2
    Dim swDocExt As ModelDocExtension
    Dim openAssem As Hashtable
    Dim tmpPath As String
    Dim tmpObj As ModelDoc2
    Dim boolstat As Boolean, stat As Boolean
    Dim strings As Object
    Dim swComponent As Component2
    Dim matefeature As Feature
    Dim MateName As String
    Dim FirstSelection As String
    Dim SecondSelection As String
    Dim strCompName As String
    Dim AssemblyTitle As String
    Dim AssemblyName As String
    Dim errors As Integer
    Dim warnings As Integer
    Dim mateError As Integer
    Dim fileName As String

```

```
    Sub Main()
```

```

        ' Open assembly
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\toolbox\lens_mount.sldasm"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors,
warnings)

        ' Set up event
        swAssemblyDoc = swModel
        openAssem = New Hashtable
        AttachEventHandlers()

        ' Get title of assembly document
        AssemblyTitle = swModel.GetTitle()

        ' Split the title into two strings using the period as the delimiter
        strings = Split(AssemblyTitle, ".")

        ' Use AssemblyName when mating the component with the assembly
        AssemblyName = strings(0)

        Debug.Print("Name of assembly: " & AssemblyName)

        boolstat = True
        Dim strCompModelname As String
        strCompModelname = "camtest.sldprt"

        ' Because the component resides in the same folder as the assembly, get
        ' the assembly's path and use it when opening the component
        tmpPath = Left(swModel.GetPathName, InStrRev(swModel.GetPathName, "\"))

        ' Open the component
        tmpObj = swApp.OpenDoc6(tmpPath + strCompModelname, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, ""

```

```

' Check to see if the file is read-only or cannot be found; display error
' messages if either
If warnings = swFileLoadWarning_e.swFileLoadWarning_ReadOnly Then
    MsgBox("This file is read-only.")
    boolstat = False
End If

If tmpObj Is Nothing Then
    MsgBox("Cannot locate the file.")
    boolstat = False
End If

' Activate the assembly so that you can add the component to it
swModel = swApp.ActivateDoc3(AssemblyTitle, True, swRebuildOnActivation_e.swUserDecision, errors)

' Add the camtest part to the assembly document
swComponent = swAssemblyDoc.AddComponent5(strCompModelname, swAddComponentConfigOptions_e.swAddComponentConfigOptions_CurrentSe

' Make the component virtual
stat = swComponent.MakeVirtual2(True)

' Get the name of the component for the mate
strCompName = swComponent.Name2()

' Create the name of the mate and the names of the planes to use for the mate
MateName = "top_coinc_" + strCompName
FirstSelection = "Top@" + strCompName & "@" + AssemblyName
SecondSelection = "Front@" + AssemblyName

swModel.ClearSelection2(True)
swDocExt = swModel.Extension

' Select the planes for the mate
boolstat = swDocExt.SelectByID2(FirstSelection, "PLANE", 0, 0, 0, True, 1, Nothing, swSelectOption_e.swSelectOptionDefault)
boolstat = swDocExt.SelectByID2(SecondSelection, "PLANE", 0, 0, 0, True, 1, Nothing, swSelectOption_e.swSelectOptionDefault)

' Add the mate
matefeature = swAssemblyDoc.AddMate5(swMateType_e.swMateCOINCIDENT, swMateAlign_e.swMateAlignALIGNED, False, 0, 0, 0, 0, 0, 0,
0, mateError)
matefeature.Name = MateName
Debug.Print("Mate added: " & matefeature.Name)

swModel.ViewZoomtofit2()

End Sub

Sub AttachEventHandlers()
    AttachSwEvents()
End Sub

Sub AttachSwEvents()
    AddHandler swAssemblyDoc.AddItemNotify, AddressOf Me.swAssemblyDoc_AddItemNotify
    AddHandler swAssemblyDoc.RenameItemNotify, AddressOf Me.swAssemblyDoc_RenameItemNotify
End Sub

Private Function swAssemblyDoc_AddItemNotify(ByVal EntityType As Integer, ByVal itemName As String) As Integer
    Debug.Print("Component added: " & itemName)
End Function

Private Function swAssemblyDoc_RenameItemNotify(ByVal EntityType As Integer, ByVal oldName As String, ByVal NewName As String) As I
    Debug.Print("Virtual component name: " & NewName)
End Function

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you
''' </summary>
Public swApp As SldWorks

End Class

```

Add Components and Transforms Example (C++ COM)

This example shows how to use transforms when adding components.

```
// -----
void APITestFunction()
{
    HRESULT          hr = S_OK;
    long              nRetVal = -1;
    CComPtr <ISldWorks> pSldWorks;
    CComPtr <IModelDoc2> pModel;
    CComQIPtr <IAssemblyDoc> pAssy;
    long              nNumComp = 2;
    BSTR              sPartPath[2] =
    {
        _T("c:/samples/Part1.SLDPRT"),
        _T("c:/samples/Part2.SLDPRT")
    };
    LPCOMPONENT       swCompArr[2] = { NULL, NULL};
    double            dTransform[32] =
    {
        1,
        0,
        0,
        0,
        1,
        0,
        0,
        0,
        1,
        0.135786163522013,
        -3.73411949685534E-02,
        -2.04999999999997E-02,
        1,
        0,
        0,
        0,
        1,
        0,
        0,
        0,
        1,
        0,
        0,
        0,
        1,
        -0.205037106918239,
        -8.82610062893077E-03,
        -5.62499999999999E-02,
        1,
        0,
        0,
        0
    };

    // Connect to SOLIDWORKS Design
    pSldWorks = TheApplication->GetSWApp();
    ASSERT(pSldWorks);
    hr = pSldWorks->get_IActiveDoc2(&pModel);
    ASSERT(pModel);
    pAssy = pModel;
    ASSERT(pAssy);
    hr = pAssy->IAddComponents2(&nNumComp, sPartPath, dTransform, swCompArr);
    ASSERT(swCompArr[0]);
    ASSERT(swCompArr[1]);
    hr = swCompArr[0]->Release();
    hr = swCompArr[1]->Release();
}
```


Add Components Example (C#)

This example shows how to add multiple components to an assembly.

```
//-----
// Preconditions:
// 1. Create a new part document.
//    a. Extrude a rectangular block and cut-extrude a diagonal section
//       off one face of the block.
//    b. Click Insert > Reference Geometry > Coordinate System.
//    c. Select a corner of the block for the origin of
//       the new coordinate system.
//    d. Select an edge for the Z axis of the coordinate system.
//    e. Click the green check mark to save the coordinate system.
//       Coordinate System1 appears in the FeatureManager design tree.
//    f. Save and minimize the part.
// 2. Replace part_path in the code with the full path name
//    of the new part.
// 3. Create a new assembly document.
//    a. Create a line segment sketch originating at some distance
//       from the default origin of the assembly document.
//    b. Click Insert > Reference Geometry > Coordinate System.
//    c. Select one end point of the line segment for the origin of
//       the new coordinate system.
//    d. Select the line segment for the X axis of the coordinate system.
//    e. Click the green check mark to save the coordinate system.
//       Coordinate System1 appears in the FeatureManager design tree.
//    f. Right-click on Coordinate System1 in the FeatureManager design tree,
//       select Feature Properties, and rename the coordinate system
//       and its description to differ from Coordinate System1.
//    g. Save the assembly and keep it open.
// 4. Change the namespace to match your project's name.
//
// Postconditions:
// Component part is inserted into the assembly such that
// the part's Coordinate System1 is coincident (no translation or rotation)
// with the assembly's default (not user-defined) coordinate system.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
namespace AddComponents3_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        AssemblyDoc assemb;
        string[] compNames = new string[1];
        double[] compXforms = new double[16];
        string[] compCoordSysNames = new string[1];
        object vcompNames;
        object vcompXforms;
        object vcompCoordSysNames;
        object vcomponents;

        public void Main()
        {
            assemb = (AssemblyDoc)swApp.ActiveDoc;

            if (((assemb != null)))
            {
                compNames[0] = "part_path";

                // Define the transformation matrix. See the IMathTransform API documentation.

                // Add a rotational diagonal unit matrix (zero rotation) to the transform
                // x-axis components of rotation
                compXforms[0] = 1.0;
                compXforms[1] = 0.0;
                compXforms[2] = 0.0;
                // y-axis components of rotation
                compXforms[3] = 0.0;
                compXforms[4] = 1.0;
                compXforms[5] = 0.0;
                // z-axis components of rotation
                compXforms[6] = 0.0;
                compXforms[7] = 0.0;
                compXforms[8] = 1.0;

                // Add a translation vector to the transform (zero translation)
                compXforms[9] = 0.0;
                compXforms[10] = 0.0;
                compXforms[11] = 0.0;

                // Add a scaling factor to the transform
                compXforms[12] = 0.0;

                // The last three elements in the transformation matrix are unused
            }
        }
    }
}
```

```

        // Register the coordinate system name for the component
        compCoordSysNames[0] = "Coordinate System1";

        // Add the components to the assembly.
        vcompNames = compNames;
        vcompXforms = compXforms;
        //vcompXforms = Nothing //also achieves zero rotation and translation of the component
        vcompCoordSysNames = compCoordSysNames;

        vcomponents = assemb.AddComponents3((vcompNames), (vcompXforms), (vcompCoordSysNames));
    }
}

public SldWorks swApp;
}
}

```

Add Components Example (VBA)

This example shows how to add multiple components to an assembly.

```

'-----
' Preconditions:
' 1. Create a new part document.
'   a. Extrude a rectangular block and cut-extrude a diagonal section
'      off one face of the block.
'   b. Click Insert > Reference Geometry > Coordinate System.
'   c. Select a corner of the block for the origin of the new coordinate system.
'   d. Select an edge for the Z axis of the coordinate system.
'   e. Click the green check mark to save the coordinate system.
'      Coordinate System1 appears in the FeatureManager design tree.
'   f. Save and minimize the part.
' 2. Replace <component_name> in the code with the full path name
'    of the new part.
' 3. Create a new assembly document.
'   a. Create a line segment sketch originating at some distance
'      from the default origin of the assembly document.
'   b. Click Insert > Reference Geometry > Coordinate System.
'   c. Select one end point of the line segment for the origin
'      of the new coordinate system.
'   d. Select the line segment for the X axis of the coordinate system.
'   e. Click the green check mark to save the coordinate system.
'      Coordinate System1 appears in the FeatureManager design tree.
'   f. Right-click on Coordinate System1 in the FeatureManager design tree,
'      select Feature Properties, and rename the coordinate system
'      and its description to differ from Coordinate System1.
'   g. Save the assembly and keep it open.
' Postconditions:
' Component part is inserted into the assembly such that
' the part's Coordinate System1 is coincident (no translation or rotation)
' with the assembly's default (not user-defined) coordinate system.
'-----

Dim swApp As SldWorks.SldWorks
Dim assem As SldWorks.Assembly
Dim compNames(0) As String
Dim compXforms(15) As Double
Dim compCoordSysNames(0) As String
Dim vcompNames As Variant
Dim vcompXforms As Variant
Dim vcompCoordSysNames As Variant
Dim vcomponents As Variant

Option Explicit

Sub main()
Set swApp = Application.SldWorks
Set assem = swApp.ActiveDoc

' For each component to be added, create a separate transform
If (Not assem Is Nothing) Then
    compNames(0) = "<component_name>"

' Define the transformation matrix. See the IMathTransform API documentation.
' Add a rotational diagonal unit matrix (zero rotation) to the transformation matrix
    compXforms(0) = 1#
    compXforms(1) = 0#
    compXforms(2) = 0#
    compXforms(3) = 0#
    compXforms(4) = 1#
    compXforms(5) = 0#
    compXforms(6) = 0#
    compXforms(7) = 0#
    compXforms(8) = 1#

' Add a translation vector to the transformation matrix
    compXforms(9) = 0#
    compXforms(10) = 0#
    compXforms(11) = 0#

' Add a scaling factor to the transform
    compXforms(12) = 1#

' The last three elements of the transformation matrix are unused

' Register the component's coordinate system name
    compCoordSysNames(0) = "Coordinate System1"

' Add the component to the assembly.
    vcompNames = compNames
    vcompXforms = compXforms
    'vcompXforms = Nothing ' also achieves zero rotation and translation of component

```

```
vcompCoordSysNames = compCoordSysNames
vcomponents = assemb.AddComponents3((vcompNames), (vcompXforms), (vcompCoordSysNames))
End If
End Sub
```

Add Components Example (VB.NET)

This example shows how to add multiple components to an assembly.

```

'-----
' Preconditions:
' 1. Create a new part document.
'   a. Extrude a rectangular block and cut-extrude a diagonal section
'      off one face of the block.
'   b. Click Insert > Reference Geometry > Coordinate System.
'   c. Select a corner of the block for the origin of the new coordinate system.
'   d. Select an edge for the Z axis of the coordinate system.
'   e. Click the green check mark to save the coordinate system.
'      Coordinate System1 appears in the FeatureManager design tree.
'   f. Save and minimize the part.
' 2. Replace <component_name> in the code with the full path name of
'    the new part.
' 3. Create a new assembly document.
'   a. Create a line segment sketch originating at some distance
'      from the default origin of the assembly document.
'   b. Click Insert > Reference Geometry > Coordinate System.
'   c. Select one end point of the line segment for the origin of
'      the new coordinate system.
'   d. Select the line segment for the X axis of the coordinate system.
'   e. Click the green check mark to save the coordinate system.
'      Coordinate System1 appears in the FeatureManager design tree.
'   f. Right-click on Coordinate System1 in the FeatureManager design tree,
'      select Feature Properties, and rename the coordinate system
'      and its description to differ from Coordinate System1.
'   g. Save the assembly.
' 4. Open an Immediate window.
' 5. Activate the assembly document.
'
' Postconditions:
' Component part is inserted into the assembly such that
' the part's Coordinate System1 is coincident (no translation or rotation)
' with the assembly's default (not user-defined) coordinate system.
'-----

```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Partial Class SolidWorksMacro
```

```

    Dim assemb As Object
    Dim compNames(0) As String
    Dim compXforms(15) As Double
    Dim compCoordSysNames(0) As String
    Dim vcompNames As Object
    Dim vcompXforms As Object
    Dim vcompCoordSysNames As Object
    Dim vcomponents As Object

```

```
Sub main()
```

```
    assemb = swApp.ActiveDoc
```

```
    If (Not assemb Is Nothing) Then
```

```
        compNames(0) = "<component_name>"
```

```
        ' Define the transformation matrix for the component. See the IMathTransform API documentation.
```

```
        ' Add a rotational diagonal unit matrix to the transformation matrix (zero rotation)
```

```

        compXforms(0) = 1.0
        compXforms(1) = 0.0
        compXforms(2) = 0.0
        compXforms(3) = 0.0
        compXforms(4) = 1.0
        compXforms(5) = 0.0
        compXforms(6) = 0.0
        compXforms(7) = 0.0
        compXforms(8) = 1.0

```

```
        ' Add a translation vector to the transformation matrix (zero translation)
```

```

        compXforms(9) = 0.0
        compXforms(10) = 0.0
        compXforms(11) = 0.0

```

```
        ' Add a scaling factor to the transformation matrix
```

```
        compXforms(12) = 0.0
```

```
        ' The last three elements of the transformation matrix are unused
```

```
        ' Add a coordinate system name assigned to each component
```

```
compCoordSysNames(0) = "Coordinate System1"
```

```
' Add the components to the assembly.
```

```
vcompNames = compNames
```

```
vcompXforms = compXforms
```

```
'vcompXforms = Nothing ' you can also pass a null transform to achieve zero rotation and translation
```

```
vcompCoordSysNames = compCoordSysNames
```

```
vcomponents = assemb.AddComponents3((vcompNames), (vcompXforms), (vcompCoordSysNames))
```

```
End If
```

```
End Sub
```

```
Public swApp As SldWorks
```

```
End Class
```

Add Configuration and Promote Child Components in BOM Example (C#)

This example shows how to add a configuration to an assembly and promote its child components one level in a BOM.

```
//-----
// Preconditions:
// 1. Specified assembly document exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified assembly document.
// 2. Adds a new configuration named Config2.
// 3. Sets the option to dissolve the assembly's
//    configuration when it appears in a BOM and
//    promote all of its child components one level.
// 4. To verify, examine the option value printed to
//    the Immediate window.
//
// NOTE: Because this assembly document is used
// elsewhere, do not save changes when closing it.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace ChildComponentDisplayInBOMCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        AssemblyDoc swAssembly;
        ModelDoc2 swModel;
        ModelDocExtension swModelDocExt;
        Configuration swConfig;
        string fileName;
        bool status;
        int errors;
        int warnings;

        public void Main()
        {
            //Open assembly document
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\advdrawings\\bowl and chute.sldasm";
            swAssembly = (AssemblyDoc)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swModel = (ModelDoc2)swAssembly;

            //Add configuration named Config2
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("bowl and chute.SLDASM", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            swModel.ClearSelection2(true);
            swConfig = ((Configuration)swModel.AddConfiguration3("Config2", "", "", (int)swConfigurationOptions2_e.swConfigOption_DoDissolveInBOM + (int)swConfigurationOptions2_e.swConfigOption_DontShowPartsInBO
M);

            //Dissolve the assembly's configuration
            //when it appears in a BOM and promote all of
            //its child components one level
            swConfig.ChildComponentDisplayInBOM = (int)swChildComponentInBOMOption_e.swChildComponent_Promote;
            Debug.Print("Child component display option in BOM: " + swConfig.ChildComponentDisplayInBOM);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Add Configuration and Promote Child Components in BOM Example (VBA)

This example shows how to add a configuration to an assembly and promote its child components one level in a BOM.

```
'-----
' Preconditions:
' 1. Specified assembly document exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified assembly document.
' 2. Adds a new configuration named Config2.
' 3. Sets the option to dissolve the assembly's
'    configuration when it appears in a BOM and
'    promote all of its child components one level.
' 4. To verify, examine the option value printed to
'    the Immediate window.
'
' NOTE: Because this assembly document is used
' elsewhere, do not save changes when closing it.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swAssembly As SldWorks.AssemblyDoc
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swConfig As SldWorks.Configuration
Dim fileName As String
Dim status As Boolean
Dim errors As Long, warnings As Long

Sub main()

    Set swApp = Application.SldWorks

    'Open assembly document
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\bowl and chute.sldasm"
    Set swAssembly = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swModel = swAssembly

    'Add configuration named Config2
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID2("bowl and chute.SLDASM", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    swModel.ClearSelection2 True
    Set swConfig = swModel.AddConfiguration3("Config2", "", "", swConfigurationOptions2_e.swConfigOption_DoDissolveInBOM +
    swConfigurationOptions2_e.swConfigOption_DontShowPartsInBOM)

    'Dissolve the assembly's configuration
    'when it appears in a BOM and promote all of
    'its child components one level
    swConfig.ChildComponentDisplayInBOM = swChildComponentInBOMOption_e.swChildComponent_Promote
    Debug.Print ("Child component display option in BOM: " & swConfig.ChildComponentDisplayInBOM)

End Sub
```

Add Configuration and Promote Child Components in BOM Example (VB.NET)

This example shows how to add a configuration to an assembly and promote its child components one level in a BOM.

```
'-----
' Preconditions:
' 1. Specified assembly document exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified assembly document.
' 2. Adds a new configuration named Config2.
' 3. Sets the option to dissolve the assembly's
'    configuration when it appears in a BOM and
'    promote all of its child components one level.
' 4. To verify, examine the option value printed to
'    the Immediate window.
'
' NOTE: Because this assembly document is used
' elsewhere, do not save changes when closing it.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swAssembly As AssemblyDoc
    Dim swModel As ModelDoc2
    Dim swModelDocExt As ModelDocExtension
    Dim swConfig As Configuration
    Dim fileName As String
    Dim status As Boolean
    Dim errors As Integer
    Dim warnings As Integer

    Sub main()

        'Open assembly document
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\ Bowl and chute.sldasm"
        swAssembly = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModel = swAssembly

        'Add configuration named Config2
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID2("Bowl and chute.SLDASM", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        swConfig = swModel.AddConfiguration3("Config2", "", "", swConfigurationOptions2_e.swConfigOption_DoDissolveInBOM + swConfigurationOptions2_e.swConfigOption_DontShowPartsInBOM)

        'Dissolve the assembly's configuration
        'when it appears in a BOM and promote all of
        'its child components one level
        swConfig.ChildComponentDisplayInBOM = swChildComponentInBOMOption_e.swChildComponent_Promote
        Debug.Print("Child component display option in BOM: " & swConfig.ChildComponentDisplayInBOM)

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class
```

Add Damper to Motion Study Example (C#)

This example shows how to add a damper to a motion study.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\wrench.sldasm.
// 2. Click View > MotionManager if the MotionStudy tabs are not visible.
// 3. Click Tools > Add-ins > SOLIDWORKS Motion to enable motion analysis.
// 4. Reference the SOLIDWORKS MotionStudy type library.
//
// Postconditions:
// 1. Adds a damper feature, LinearDamper1, between the grips of the wrench.
// 2. Examine the Motion Analysis to verify.
//
// NOTE: Because the assembly is used elsewhere, do not save changes.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using System.Windows.Forms;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.swmotionstudy;
using System.Runtime.InteropServices;
namespace SimulationDamperFeatureData_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            MotionStudyManager swMotionMgr = default(MotionStudyManager);
            MotionStudy swMotionStudy1 = default(MotionStudy);
            SimulationDamperFeatureData swDamperFeat = default(SimulationDamperFeatureData);
            bool boolstatus = false;
            Feature swFeat = default(Feature);
            SelectionMgr swSelMgr = default(SelectionMgr);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = swModel.Extension;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            // Get the MotionManager
            swMotionMgr = (MotionStudyManager)swModelDocExt.GetMotionStudyManager();

            if ((swMotionMgr == null))
            {
                return;
            }

            // Get and activate "MotionStudy1_Distance=0.5in"
            swMotionStudy1 = swMotionMgr.GetMotionStudy("MotionStudy1_Distance=0.5in");

            if ((swMotionStudy1 == null))
            {
                MessageBox.Show("MotionStudy1_Distance=0.5in is not available.");
                return;
            }

            // Activate swMotionStudy1
            swMotionStudy1.Activate();

            // Define Damper feature
            swDamperFeat =
(SimulationDamperFeatureData)swMotionStudy1.CreateDefinition((int)swFeatureNameID_e.swFmAEMLinearDamper);

            if (swDamperFeat == null)
            {
                Debug.Print("ERROR: Creation of Damper feature data object failed.");
                return;
            }

            // Select the faces for the Damper's endpoints
            Face2 swFace1 = default(Face2);
            Face2 swFace2 = default(Face2);

            swModel.ShowNamedView2("Left", 3);
            swModel.ViewZoomtofit2();
            boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.03344586330968, 0.0525345575174, 0, true, 0, null, 0);
            swFace1 = (Face2)swSelMgr.GetSelectedObject6(1, -1);
            boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.02244533711473, 0.0131288302002, 0.0002238961779386, true, 0,
null, 0);
```

```

swFace2 = (Face2)swSelMgr.GetSelectedObject6(2, -1);

// Set Damper's characteristics
swDamperFeat.SetEndPoint1(swFace1, swFace2);

// Create Damper feature
swFeat = (Feature)swMotionStudy1.CreateFeature(swDamperFeat);

if (swFeat == null)
{
    Debug.Print(" ERROR: Creation of the Damper feature failed.");
}
else
{
    Debug.Print("Type of Damper as defined in swSimulationDamperType_e: " + swDamperFeat.Type);
}
}

public SldWorks swApp;
}
}

```

Add Damper to Motion Study Example (VBA)

This example shows how to add a damper to a motion study.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\wrench.sldasm.
' 2. Click View > MotionManager if the MotionStudy tabs are not visible.
' 3. Click Tools > Add-ins > SOLIDWORKS Motion to enable motion analysis.
' 4. Reference the SOLIDWORKS MotionStudy type library.
'
' Postconditions:
' 1. Adds a damper feature, LinearDamper1, between the grips of the wrench.
' 2. Examine the Motion Analysis to verify.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----

```

Option Explicit

Sub main()

```

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swModelDocExt As SldWorks.ModelDocExtension
    Dim swMotionMgr As MotionStudyManager
    Dim swMotionStudy1 As MotionStudy
    Dim swDamperFeat As SldWorks.SimulationDamperFeatureData
    Dim boolstatus As Boolean
    Dim swFeat As SldWorks.Feature
    Dim swSelMgr As SelectionMgr

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension
    Set swSelMgr = swModel.SelectionManager

    ' Get the MotionManager
    Set swMotionMgr = swModelDocExt.GetMotionStudyManager()

    If (swMotionMgr Is Nothing) Then
        End
    End If

    ' Get and activate "MotionStudy1_Distance=0.5in"
    Set swMotionStudy1 = swMotionMgr.GetMotionStudy("MotionStudy1_Distance=0.5in")

    If (swMotionStudy1 Is Nothing) Then
        MsgBox "MotionStudy1_Distance=0.5in is not available."
        End
    End If

    ' Activate swMotionStudy1
    swMotionStudy1.Activate

    ' Define Damper feature
    Set swDamperFeat = swMotionStudy1.CreateDefinition(swFmAEMLinearDamper)

    If swDamperFeat Is Nothing Then
        Debug.Print "ERROR: Creation of Damper feature data object failed."
        Exit Sub
    End If

    ' Select the faces for the Damper's endpoints
    Dim swFace1 As Face2, swFace2 As Face2
    swModel.ShowNamedView2 "**Left", 3
    swModel.ViewZoomtofit2
    boolstatus = swModelDocExt.SelectByID("", "FACE", 0.03344586330968, 0.0525345575174, 0, True, 0, Nothing, 0)
    Set swFace1 = swSelMgr.GetSelectedObject6(1, -1)
    boolstatus = swModelDocExt.SelectByID("", "FACE", 0.02244533711473, 0.0131288302002, 2.238961779386E-04, True, 0, Nothing, 0)
    Set swFace2 = swSelMgr.GetSelectedObject6(2, -1)

    ' Set Damper's characteristics
    swDamperFeat.SetEndpoints swFace1, swFace2

    ' Create Damper feature
    Set swFeat = swMotionStudy1.CreateFeature(swDamperFeat)

    If swFeat Is Nothing Then
        Debug.Print " ERROR: Creation of the Damper feature failed."
    Else
        Debug.Print "Type of Damper as defined in swSimulationDamperType_e: " & swDamperFeat.Type
    End If

```

End Sub

Add Damper to Motion Study Example (VB.NET)

This example shows how to add a damper to a motion study.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\wrench.sldasm.
' 2. Click View > MotionManager if the MotionStudy tabs are not visible.
' 3. Click Tools > Add-ins > SOLIDWORKS Motion to enable motion analysis.
' 4. Reference the SOLIDWORKS MotionStudy type library.
'
' Postconditions:
' 1. Adds a damper feature, LinearDamper1, between the grips of the wrench.
' 2. Examine the Motion Analysis to verify.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swmotionstudy
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swMotionMgr As MotionStudyManager
        Dim swMotionStudy1 As MotionStudy
        Dim swDamperFeat As SimulationDamperFeatureData
        Dim boolstatus As Boolean
        Dim swFeat As Feature
        Dim swSelMgr As SelectionMgr

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension
        swSelMgr = swModel.SelectionManager

        ' Get the MotionManager
        swMotionMgr = swModelDocExt.GetMotionStudyManager()

        If (swMotionMgr Is Nothing) Then
            Exit Sub
        End If

        ' Get and activate "MotionStudy1_Distance=0.5in"
        swMotionStudy1 = swMotionMgr.GetMotionStudy("MotionStudy1_Distance=0.5in")

        If (swMotionStudy1 Is Nothing) Then
            MsgBox("MotionStudy1_Distance=0.5in is not available.")
            Exit Sub
        End If

        ' Activate swMotionStudy1
        swMotionStudy1.Activate()

        ' Define Damper feature
        swDamperFeat = swMotionStudy1.CreateDefinition(swFeatureNameID_e.swFmAEMLinearDamper)

        If swDamperFeat Is Nothing Then
            Debug.Print("ERROR: Creation of Damper feature data object failed.")
            Exit Sub
        End If

        ' Select the faces for the Damper's endpoints
        Dim swFace1 As Face2, swFace2 As Face2

        swModel.ShowNamedView2("Left", 3)
        swModel.ViewZoomtofit2()
        boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.03344586330968, 0.0525345575174, 0, True, 0, Nothing, 0)
        swFace1 = swSelMgr.GetSelectedObject6(1, -1)
        boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.02244533711473, 0.0131288302002, 0.0002238961779386, True, 0, Nothing,
0)
        swFace2 = swSelMgr.GetSelectedObject6(2, -1)

        ' Set Damper's characteristics
        swDamperFeat.SetEndPoints(swFace1, swFace2)

        ' Create Damper feature
        swFeat = swMotionStudy1.CreateFeature(swDamperFeat)

        If swFeat Is Nothing Then
            Debug.Print(" ERROR: Creation of the Damper feature failed.")
        Else
            Debug.Print("Type of Damper as defined in swSimulationDamperType_e: " & swDamperFeat.Type)
        End If
    End Sub
End Class

```

End Sub

Public swApp As SldWorks

End Class

Add Decal Example (C#)

This example shows how to add a decal to a selected face on a part.

```
//-----
// Preconditions:
// 1. Verify that the specified part document to open exists.
// 2. Verify that the specified decal files exist.
// 3. Verify that a ray-trace rendering engine is active.
// 4. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified part document.
// 2. Selects the face where to apply a decal.
// 3. Applies a decal to the selected face.
// 4. Gets the properties of the decal and prints them
//    to the Immediate window
// 5. Examine the graphics area and the Immediate window.
//
// NOTE: Because the part is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace FaceDecalPropertiesCSharp.csproj
{
    public partial class SolidWorksMacro
    {

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            Face2 swFace = default(Face2);
            Decal swDecal = default(Decal);
            RenderMaterial swMaterial = default(RenderMaterial);
            object[] swFaceDecalPropertiesArray = null;
            FaceDecalProperties swFaceDecalProperties = default(FaceDecalProperties);
            bool status = false;
            string strname = null;
            int nDecalID = 0;
            int i = 0;
            int errors = 0;
            int warnings = 0;
            string fileName = null;
            int nbrDecals = 0;

            //Open part document and select face where to
            //apply a decal
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\tolanalyst\\minimum_clearance\\top_plate.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref error
s, ref warnings);
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("", "FACE", -0.00374778769440809, 0.0609129688728558, 0.0160000000000196, false, 0, null, 0);
            swFace = (Face2)swSelMgr.GetSelectedObject6(1, -1);
            swModel.ClearSelection2(true);

            //Create the decal
            swDecal = (Decal)swModelDocExt.CreateDecal();
            swMaterial = (RenderMaterial)swDecal;
            status = swMaterial.AddEntity(swFace);
            strname = "C:\\Program Files\\SOLIDWORKS Corp\\SOLIDWORKS\\data\\graphics\\Decals\\Logos\\sw.p2d";
            swMaterial.FileName = strname;
            strname = "C:\\Program Files\\SOLIDWORKS Corp\\SOLIDWORKS\\data\\graphics\\Decals\\Logos\\sw.bmp";
            swMaterial.TextureFilename = strname;
            swMaterial.MappingType = 0;
            swMaterial.FixedAspectRatio = false;
            swMaterial.FitHeight = true;
            swMaterial.FitWidth = true;
            status = swModelDocExt.AddDecal(swDecal, out nDecalID);

            //Rebuild the model
            swModelDocExt.Rebuild((int)swRebuildOptions_e.swRebuildAll);

            //Get decal properties
            status = swModelDocExt.SelectByID2("", "FACE", -0.00374778769440809, 0.0609129688728558, 0.0160000000000196, false, 0, null, 0);
            swFace = (Face2)swSelMgr.GetSelectedObject6(1, -1);
            nbrDecals = swFace.GetDecalsCount();
            Debug.Print("Number of decals applied to this face: " + nbrDecals);
            if (nbrDecals > 0)
            {
                swFaceDecalPropertiesArray = (object[])swFace.GetAllDecalProperties();
                for (i = 0; i < swFaceDecalPropertiesArray.Length; i++)
                {
                    swFaceDecalProperties = (FaceDecalProperties)swFaceDecalPropertiesArray[i];
                    Debug.Print("Decal " + (i + 1) + "'s texture:");
                    Debug.Print("  Angle: " + swFaceDecalProperties.TextureAngle);
                    Debug.Print("  Angle UV: " + swFaceDecalProperties.TextureAngleUV);
                    Debug.Print("  File: ");
                    Debug.Print("    Name: " + swFaceDecalProperties.TextureFilename);
                    Debug.Print("    ID: " + swFaceDecalProperties.TextureFilenameID);
                    Debug.Print("    ID: " + swFaceDecalProperties.TextureID);
                    Debug.Print("    Map ID: " + swFaceDecalProperties.TextureMapID);
                    Debug.Print("    Mirrored? " + swFaceDecalProperties.TextureMirrored);
                    Debug.Print("    Render mode (0 = Image; 1 = Luminance): " + swFaceDecalProperties.TextureRenderMode);
                    Debug.Print("    Translation: ");
                    Debug.Print("      U: " + swFaceDecalProperties.TextureTranslationU);
                    Debug.Print("      V: " + swFaceDecalProperties.TextureTranslationV);
                    Debug.Print("      X: " + swFaceDecalProperties.TextureTranslationX);
                    Debug.Print("      Y: " + swFaceDecalProperties.TextureTranslationY);
                    Debug.Print("    UV scale: ");
                    Debug.Print("      U: " + swFaceDecalProperties.TextureUScale);
                    Debug.Print("      V: " + swFaceDecalProperties.TextureVScale);
                }
            }
        }
    }
}
```

```
    }  
}  
/// <summary>  
/// The Sldworks swApp variable is pre-assigned for you.  
/// </summary>  
public SldWorks swApp;  
}
```

Add Decal Example (VBA)

This example shows how to add a decal to a selected face on a part.

```

'-----
' Preconditions:
' 1. Verify that the specified part document to open exists.
' 2. Verify that the specified decal files exist.
' 3. Verify that a ray-trace rendering engine is active.
' 4. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part document.
' 2. Selects the face where to apply a decal.
' 3. Applies a decal to the selected face.
' 4. Gets the properties of the decal and prints them
'    to the Immediate window.
' 5. Examine the graphics area and the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----

Option Explicit
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFace As SldWorks.Face2
Dim swDecal As SldWorks.Decal
Dim swMaterial As SldWorks.RenderMaterial
Dim swFaceDecalPropertiesArray As Variant
Dim swFaceDecalProperties As SldWorks.FaceDecalProperties
Dim status As Boolean
Dim strname As String
Dim nDecalID As Long
Dim i As Long
Dim errors As Long
Dim warnings As Long
Dim fileName As String
Dim nbrDecals As Long

Sub main()

    Set swApp = Application.SldWorks

    'Open part document and select face where to
    'apply a decal
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\tolanalyst\minimum_clearance\top_plate.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swSelMgr = swModel.SelectionManager
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID("", "FACE", -3.74778769440809E-03, 6.09129688728558E-02, 1.60000000000196E-02, False, 0, Nothing, 0)
    Set swFace = swSelMgr.GetSelectedObject6(1, -1)
    swModel.ClearSelection2 True

    'Create the decal
    Set swDecal = swModelDocExt.CreateDecal
    Set swMaterial = swDecal
    status = swMaterial.AddEntity(swFace)
    strname = "C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\data\graphics\Decals\Logos\sw.p2d"
    swMaterial.fileName = strname
    strname = "C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\data\graphics\Decals\Logos\sw.bmp"
    swMaterial.TextureFilename = strname
    swMaterial.MappingType = 0
    swMaterial.FixedAspectRatio = False
    swMaterial.FitHeight = True
    swMaterial.FitWidth = True
    status = swModelDocExt.AddDecal(swDecal, nDecalID)

    'Rebuild the model
    Call swModelDocExt.Rebuild(swRebuildAll)

    'Get decal properties
    status = swModelDocExt.SelectByID("", "FACE", -3.74778769440809E-03, 6.09129688728558E-02, 1.60000000000196E-02, False, 0, Nothing, 0)
    Set swFace = swSelMgr.GetSelectedObject6(1, -1)
    nbrDecals = swFace.GetDecalsCount
    Debug.Print "Number of decals applied to this face: " & nbrDecals
    If nbrDecals > 0 Then
        swFaceDecalPropertiesArray = swFace.GetAllDecalProperties
        For i = 0 To UBound(swFaceDecalPropertiesArray)
            Set swFaceDecalProperties = swFaceDecalPropertiesArray(i)
            Debug.Print "Decal " & i + 1 & "'s texture:"
            Debug.Print "  Angle: " & swFaceDecalProperties.TextureAngle
            Debug.Print "  Angle UV: " & swFaceDecalProperties.TextureAngleUV
            Debug.Print "  File: "
            Debug.Print "    Name: " & swFaceDecalProperties.TextureFilename
            Debug.Print "    ID: " & swFaceDecalProperties.TextureFilenameID
            Debug.Print "  ID: " & swFaceDecalProperties.TextureID
            Debug.Print "  Map ID: " & swFaceDecalProperties.TextureMapID
            Debug.Print "  Mirrored? " & swFaceDecalProperties.TextureMirrored
            Debug.Print "  Render mode (0 = Image; 1 = Luminance): " & swFaceDecalProperties.TextureRenderMode
            Debug.Print "  Translation: "
            Debug.Print "    U: " & swFaceDecalProperties.TextureTranslationU
            Debug.Print "    V: " & swFaceDecalProperties.TextureTranslationV
            Debug.Print "    X: " & swFaceDecalProperties.TextureTranslationX
            Debug.Print "    Y: " & swFaceDecalProperties.TextureTranslationY
        Next i
    End If
End Sub

```

```
        Debug.Print "    UV scale: "
        Debug.Print "        U: " & swFaceDecalProperties.TextureUScale
        Debug.Print "        V: " & swFaceDecalProperties.TextureVScale
    Next i
End If
End Sub
```

Add Decal Example (VB.NET)

This example shows how to add a decal to a selected face on a part.

```

Preconditions:
1. Verify that the specified part document to open exists.
2. Verify that the specified decal files exist.
3. Verify that a ray-trace rendering engine is active.
4. Open the Immediate window.

Postconditions:
1. Opens the specified part document.
2. Selects the face where to apply a decal.
3. Applies a decal to the selected face.
4. Gets the properties of the decal and prints them
   to the Immediate window.
5. Examine the graphics area and the Immediate window.

NOTE: Because the part is used elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swModelDocExt As ModelDocExtension
        Dim swFace As Face2
        Dim swDecal As Decal
        Dim swMaterial As RenderMaterial
        Dim swFaceDecalPropertiesArray() As Object
        Dim swFaceDecalProperties As FaceDecalProperties
        Dim status As Boolean
        Dim strname As String
        Dim nDecalID As Integer
        Dim i As Integer
        Dim errors As Integer
        Dim warnings As Integer
        Dim fileName As String
        Dim nbrDecals As Integer

        'Open part document and select face where to
        'apply a decal
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS_2018\samples\tutorial\tolanalyst\minimum_clearance\top_plate.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swSelMgr = swModel.SelectionManager
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID2("", "FACE", -0.00374778769440809, 0.0609129688728558, 0.01600000000000196, False, 0, Nothing, 0)
        swFace = swSelMgr.GetSelectedObject6(1, -1)
        swModel.ClearSelection2(True)

        'Create the decal
        swDecal = swModelDocExt.CreateDecal
        swMaterial = swDecal
        status = swMaterial.AddEntity(swFace)
        strname = "C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\data\graphics\Decals\Logos\sw.p2d"
        swMaterial.FileName = strname
        strname = "C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\data\graphics\Decals\Logos\sw.bmp"
        swMaterial.TextureFilename = strname
        swMaterial.MappingType = 0
        swMaterial.FixedAspectRatio = False
        swMaterial.FitHeight = True
        swMaterial.FitWidth = True
        status = swModelDocExt.AddDecal(swDecal, nDecalID)

        'Rebuild the model
        Call swModelDocExt.Rebuild(swRebuildOptions_e.swRebuildAll)

        'Get decal properties
        status = swModelDocExt.SelectByID2("", "FACE", -0.00374778769440809, 0.0609129688728558, 0.01600000000000196, False, 0, Nothing, 0)
        swFace = swSelMgr.GetSelectedObject6(1, -1)
        nbrDecals = swFace.GetDecalsCount
        Debug.Print("Number of decals applied to this face: " & nbrDecals)
        If nbrDecals > 0 Then
            swFaceDecalPropertiesArray = swFace.GetAllDecalProperties
            For i = 0 To UBound(swFaceDecalPropertiesArray)
                swFaceDecalProperties = swFaceDecalPropertiesArray(i)
                Debug.Print("Decal " & i + 1 & "'s texture:")
                Debug.Print("  Angle: " & swFaceDecalProperties.TextureAngle)
                Debug.Print("  Angle UV: " & swFaceDecalProperties.TextureAngleUV)
                Debug.Print("  File: ")
                Debug.Print("    Name: " & swFaceDecalProperties.TextureFilename)
                Debug.Print("    ID: " & swFaceDecalProperties.TextureFilenameID)
                Debug.Print("    ID: " & swFaceDecalProperties.TextureID)
                Debug.Print("    Map ID: " & swFaceDecalProperties.TextureMapID)
                Debug.Print("    Mirrored? " & swFaceDecalProperties.TextureMirrored)
                Debug.Print("    Render mode (0 = Image; 1 = Luminance): " & swFaceDecalProperties.TextureRenderMode)
                Debug.Print("    Translation: ")
                Debug.Print("      U: " & swFaceDecalProperties.TextureTranslationU)
                Debug.Print("      V: " & swFaceDecalProperties.TextureTranslationV)
                Debug.Print("      X: " & swFaceDecalProperties.TextureTranslationX)
                Debug.Print("      Y: " & swFaceDecalProperties.TextureTranslationY)
                Debug.Print("    UV scale: ")
                Debug.Print("      U: " & swFaceDecalProperties.TextureUScale)
                Debug.Print("      V: " & swFaceDecalProperties.TextureVScale)
            Next i
        End If

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>

```

Public swApp As SldWorks

End Class

Add Derived Configurations Example (C#)

This example shows how to add a derived configuration for each existing configuration.

```
//-----
// Preconditions: Open a part or assembly.
//
// Postconditions: For each configuration, adds and selects a derived
// configuration.
//
// NOTE: IConfigurationManager::AddConfiguration returns
// Nothing or null if the new configuration already exists.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
namespace SelectConfiguration_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            string[] vConfigNameArr = null;

            Configuration swConf = default(Configuration);
            ConfigurationManager swConfMgr = default(ConfigurationManager);
            Configuration swDerivConf = default(Configuration);
            bool bRet = false;
            int i;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swConfMgr = swModel.ConfigurationManager;
            swConf = swConfMgr.ActiveConfiguration;

            vConfigNameArr = (string[])swModel.GetConfigurationNames();

            for (i = 0; i <= vConfigNameArr.GetUpperBound(0); i++) {
                swConf = (Configuration)swModel.GetConfigurationByName(vConfigNameArr[i]);

                // Nothing or null if (derived) configuration already exists
                swDerivConf = swConfMgr.AddConfiguration(swConf.Name + " Derived", "Derived comment", "Derived alternate name", 1,
                swConf.Name, "Derived description");
                bRet = swDerivConf.Select2(true, null);
            }
        }

        public SldWorks swApp;
    }
}
```

Add Derived Configurations Example (VBA)

This example shows how to add a derived configuration for each existing configuration.

```

'-----
' Preconditions: Open a part or assembly.
'
' Postconditions: For each configuration, adds and selects
' a derived configuration.
'
' NOTE: IConfigurationManager::AddConfiguration returns
' Nothing or null if the new configuration already exists.
'-----
Option Explicit
Sub main()
    Dim swApp                As SldWorks.SldWorks
    Dim swModel              As SldWorks.ModelDoc2
    Dim vConfigNameArr       As Variant
    Dim vConfigName          As Variant
    Dim swConf               As SldWorks.Configuration
    Dim swConfMgr             As SldWorks.ConfigurationManager
    Dim swDerivConf           As SldWorks.Configuration
    Dim bRet                  As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swConfMgr = swModel.ConfigurationManager
    Set swConf = swConfMgr.ActiveConfiguration

    vConfigNameArr = swModel.GetConfigurationNames

    For Each vConfigName In vConfigNameArr
        Set swConf = swModel.GetConfigurationByName(vConfigName)
        ' Nothing or null if (derived) configuration already exists
        Set swDerivConf = swConfMgr.AddConfiguration(swConf.Name + " Derived", "Derived comment", "Derived alternate name", 1,
        swConf.Name, "Derived description")
        bRet = swDerivConf.Select2(True, Nothing)
    Next

End Sub

```

Add Derived Configurations Example (VB.NET)

This example shows how to add a derived configuration for each existing configuration.

```

'-----
' Preconditions: Open a part or assembly.
'
' Postconditions: For each configuration, adds and selects a derived
' configuration.
'
' NOTE: IConfigurationManager::AddConfiguration returns
' Nothing or null if the new configuration already exists.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Sub main()

        Dim swModel As ModelDoc2
        Dim vConfigNameArr As Object
        Dim vConfigName As Object
        Dim swConf As Configuration
        Dim swConfMgr As ConfigurationManager
        Dim swDerivConf As Configuration
        Dim bRet As Boolean

        swModel = swApp.ActiveDoc
        swConfMgr = swModel.ConfigurationManager
        swConf = swConfMgr.ActiveConfiguration

        vConfigNameArr = swModel.GetConfigurationNames

        For Each vConfigName In vConfigNameArr
            swConf = swModel.GetConfigurationByName(vConfigName)

            ' Nothing or null if (derived) configuration already exists
            swDerivConf = swConfMgr.AddConfiguration(swConf.Name + " Derived", "Derived comment", "Derived alternate name", 1,
            swConf.Name, "Derived description")
            bRet = swDerivConf.Select2(True, Nothing)
        Next

    End Sub

    Public swApp As SldWorks

End Class

```

Add Distance Mates Example (VBA)

This example shows how to add distance mate to an assembly.

```
'-----
' Preconditions: Open public_documents\samples\tutorial\api\assem20.sldasm.
'
' Postconditions:
' 1. Selects Front Plane of the block20<1> component. Click OK to
'   to close the message box.
' 2. Selects Front Plane of the cylinder20<1> component. Click OK
'   to close the message box.
' 3. Adds a distance mate using the Front Plane of each component.
' 4. Displays the number of mates created. Click OK to close the
'   message box.
' 5. Expand Mates in the FeatureManager design tree and examine
'   the graphics area to verify steps 3 and 4.
'
' NOTE: Because this assembly is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swAssy As SldWorks.AssemblyDoc
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim longstatus As Long
Dim mateFeature As Object
Dim mateSelMark As Long
Dim numberOfMatesCreated As Long
Dim status As Boolean
Dim strMessage As String

Sub main()

    Set swApp = Application.SldWorks
    Set swAssy = swApp.ActiveDoc
    Set swModel = swAssy
    Set swModelDocExt = swModel.Extension
    mateSelMark = 1
    numberOfMatesCreated = 0

    ' Insert distance mate
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID("Front Plane@block20-1@assem20", "PLANE", 0, 0, 0, True, mateSelMark, Nothing,
swSelectOption_e.swSelectOptionDefault)
    If status Then MsgBox "Front Plane@block20-1@assem20 selected!"
    status = swModelDocExt.SelectByID("Front Plane@cylinder20-1@assem20", "PLANE", 0, 0, 0, True, mateSelMark, Nothing,
swSelectOption_e.swSelectOptionDefault)
    If status Then MsgBox "Front Plane@cylinder20-1@assem20 selected!"
    Set mateFeature = swAssy.AddMate5(swMateType_e.swMateDISTANCE, swMateAlign_e.swMateAlignALIGNED, True, 0.254, 0.254, 0.254, 0.0254, 0.0254, 0,
0.5235987755983, 0.5235987755983, False, False, 0, longstatus)
    If mateFeature Is Nothing Then
        MsgBox "Distance mate failed! "
    Else
        numberOfMatesCreated = numberOfMatesCreated + 1
    End If

    swModel.ClearSelection2 True

    strMessage = "Number of mates created = "
    strMessage = strMessage + CStr(numberOfMatesCreated)
    MsgBox strMessage

    swModel.EditRebuild3

End Sub
```

Add Equation and Evaluate All Example (C#)

This example shows how to use EquationMgr interfaces to add an equation to a model and delay evaluation until all equations have been added.

```
//-----
// Preconditions:
// 1. Open any model document.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Rebuild the model.
// 2. Observe the 26 (A-Z) new equations in the Equations folder in the
//    FeatureManager design tree.
// 3. Observe the near-zero evaluation time for each equation in the
//    Immediate Window, demonstrating that the evaluations were delayed.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;

namespace AddEquationAndEvaluateAll_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            swModel = (ModelDoc2)swApp.ActiveDoc;
            EquationMgr MyEqu = default(EquationMgr);

            int Index = 0;
            long lValue = 0;
            long evalStatus = 0;

            System.DateTime t1 = default(System.DateTime);
            System.DateTime t2 = default(System.DateTime);

            long i = 0;
            i = 0;
            MyEqu = swModel.GetEquationMgr();
            if (MyEqu.GetCount() > 0)
            {
                while ((i < 26))
                {
                    MyEqu.Delete(0);
                    i = i + 1;
                }
            }
            Debug.Print("Delaying evaluation of equations until the end...");
            for (Index = System.Convert.ToInt32('A'); Index <= System.Convert.ToInt32('Z'); Index++)
            {
                t1 = DateTime.Now;
                //Delay solving each equation until after all equations are added
                //(set solve parameter to false)
                //FeatureManager design tree not updated
                lValue = MyEqu.Add2(Index, "\"" + System.Convert.ToChar(Index) + "\"=" + Index, false);

                t2 = DateTime.Now;
                Debug.Print("Time of evaluation for character " + System.Convert.ToChar(Index) + ": ");
                TimeSpan t3 = t2 - t1;
                Debug.Print(t3.ToString());
            }
            Debug.Print("Number of equations added to EquationMgr is " + MyEqu.GetCount());
            //Solve all equations after they have been added
            //FeatureManager design tree updated
            Debug.Print("Evaluating all equations...");
            evalStatus = MyEqu.EvaluateAll();
            Debug.Print("Finished Add2 and EvaluateAll");
            MyEqu = null;
            swModel = null;
            //swApp.CloseAllDocuments True
            swApp = null;
        }
    }
}
```

```
}  
    public StdWorks swApp;  
}  
}
```

Add Equation and Evaluate All Example (VBA)

This example shows how to use IEquationMgr interface to add an equation to a model and delay evaluation until all equations are added.

```

'-----
' Preconditions:
' 1. Open any model document.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Rebuild the model.
' 2. Observe the 26 (A-Z) new equations in the Equations folder in the
'    FeatureManager design tree.
' 3. Observe the near-zero evaluation time for each equation in the
'    Immediate Window, demonstrating that the evaluations were delayed.
'-----
Option Explicit

#Const ccTIMER = 1

Public Sub Main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swPart As SldWorks.PartDoc

    Set swApp = Application.SldWorks

    Dim errors As Long
    Dim warnings As Long

    Set swModel = swApp.ActiveDoc
    Dim MyEqu As SldWorks.EquationMgr

    Dim Index As Integer
    Dim lValue As Long
    Dim evalStatus As Long

    #If ccTIMER = 1 Then
        Dim t1 As Single
        Dim t2 As Single
    #Else
        Dim t1 As Date
        Dim t2 As Date
    #End If

    Dim i As Long
    i = 0
    Set MyEqu = swModel.GetEquationMgr
    If MyEqu.GetCount > 0 Then
        Do While (i < 26)
            MyEqu.Delete (0)
            i = i + 1
        Loop
    End If

    For Index = Asc("A") To Asc("Z")
        #If ccTIMER = 1 Then
            t1 = Timer
        #Else
            t1 = Now
        #End If

        'Delay solving each equation until after all equations are
        'added (set solve parameter to false)
        'FeatureManager design tree not updated
        lValue = MyEqu.Add2(Index, """" & Chr$(Index) & """" & Index, False)

        #If ccTIMER = 1 Then
            t2 = Timer
        #Else
            t2 = Now
        #End If

        Debug.Print "Time of evaluation: "
        Debug.Print (t2 - t1)
    Next Index

    Debug.Print "Number of equations added to EquationMgr is " & MyEqu.GetCount

    'Solve all equations after they have been added
    'FeatureManager design tree updated
    evalStatus = MyEqu.EvaluateAll

    Debug.Print "Finished Add2 and EvaluateAll"

    Set MyEqu = Nothing
    Set swModel = Nothing
    Set swApp.CloseAllDocuments True
    Set swApp = Nothing

End Sub

```

Add Equation and Evaluate All Example (VB.NET)

This example shows how to use IEquationMgr interfaces to add an equation to a model and delay evaluation until all equations are added.

```

'-----
' Preconditions:
' 1. Open any model document.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Rebuild the model.
' 2. Observe the 26 (A-Z) new equations in the Equations folder in the
'    FeatureManager design tree.
' 3. Observe the near-zero evaluation time for each equation in the
'    Immediate Window, demonstrating that the evaluations were delayed.
'-----
#Const ccTIMER = 1

Public Sub Main()
    Dim swModel As ModelDoc2
    swModel = swApp.ActiveDoc

    Dim MyEqu As EquationMgr
    Dim Index As Integer
    Dim IValue As Long
    Dim evalStatus As Long

    #If ccTIMER = 1 Then
        Dim t1 As Single
        Dim t2 As Single
    #Else
        Dim t1 As Date
        Dim t2 As Date
    #End If

    Dim i As Long
    i = 0

    MyEqu = swModel.GetEquationMgr
    If MyEqu.GetCount > 0 Then
        Do While (i < 26)
            MyEqu.Delete(0)
            i = i + 1
        Loop
    End If

    Debug.Print("Delaying evaluation of equations until the end...")

    For Index = Asc("A") To Asc("Z")
        #If ccTIMER = 1 Then
            t1 = Timer
        #Else
            t1 = Now
        #End If

        'Delay solving each equation until after all equations are
        'added (set solve parameter to false)
        'FeatureManager design tree not updated

        IValue = MyEqu.Add2(Index, "" & Microsoft.VisualBasic.Chr(Index) & "" & Index, False)

        #If ccTIMER = 1 Then
            t2 = Timer
        #Else
            t2 = Now
        #End If

        Debug.Print("Time of evaluation for character " & Microsoft.VisualBasic.Chr(Index) & ": ")
        Debug.Print(t2 - t1)

    Next Index

    Debug.Print("Number of equations added to EquationMgr is " & MyEqu.GetCount)

    'Solve all equations after they have been added
    'FeatureManager design tree updated

    Debug.Print("Evaluating all equations...")
    evalStatus = MyEqu.EvaluateAll()
    Debug.Print("Finished Add2 and EvaluateAll")

    MyEqu = Nothing
    swModel = Nothing
    swApp.CloseAllDocuments True
    swApp = Nothing

End Sub

Public swApp As SldWorks

End Class

```

Add and Modify Equations Example (C#)

This example shows how to add equations to multiple configurations of a part created in SOLIDWORKS 2014 or later.

```
//-----
// Preconditions: Open a part created in SOLIDWORKS 2014 or later that has a
// Boss-Extrude1 feature and multiple configurations.
//
// Postconditions:
// 1. Adds two equations to all configurations.
// 2. Modifies the equation at index 1.
// 3. Click Tools > Equations and examine each configuration in the dialog.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
namespace Add3_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 Part = default(ModelDoc2);
            EquationMgr swEquationMgr = default(EquationMgr);
            long longEquation = 0;

            Part = (ModelDoc2)swApp.ActiveDoc;

            swEquationMgr = Part.GetEquationMgr();
            if (swEquationMgr == null)
                ErrorMsg(swApp, "Failed to get the equation manager");

            //Add a global variable assignment at index, 0, to all configurations
            longEquation = swEquationMgr.Add3(0, "\"A\" = 2in", true, (int)swInConfigurationOpts_e.swAllConfiguration, null);
            if (longEquation != 0)
                ErrorMsg(swApp, "Failed to add a global variable assignment");

            //Add a dimension equation at index, 1, to all configurations
            longEquation = swEquationMgr.Add3(1, "\"D1@Boss-Extrude1\" = 0.05in", true,
            (int)swInConfigurationOpts_e.swAllConfiguration, null);
            if (longEquation != 1)
                ErrorMsg(swApp, "Failed to add a dimension equation");

            //Modify dimension equation at index, 1, in all configurations
            longEquation = swEquationMgr.SetEquationAndConfigurationOption(1, "\"D1@Boss-Extrude1\" = 0.07in",
            (int)swInConfigurationOpts_e.swAllConfiguration, null);
            if (longEquation != 1)
                ErrorMsg(swApp, "Failed to modify a dimension equation");

        }

        public void ErrorMsg(SldWorks swApp, string Message)
        {
            swApp.SendMsgToUser2(Message, 0, 0);
            swApp.RecordLine("**** WARNING - General");
            swApp.RecordLine("**** " + Message);
            swApp.RecordLine("");
        }

        public SldWorks swApp;
    }
}
```

Add and Modify Equations Example (VBA)

This example shows how to add equations to multiple configurations of a part created in SOLIDWORKS 2014 or later.

```

'-----
' Preconditions: Open a part created in SOLIDWORKS 2014 or later that has a
' Boss-Extrude1 feature and multiple configurations.
'
' Postconditions:
' 1. Adds two equations to all configurations.
' 2. Modifies the equation at index 1.
' 3. Click Tools > Equations and examine each configuration in the dialog.
'-----
Option Explicit

Sub main()

    Dim SwApp          As SldWorks.SldWorks
    Dim Part           As SldWorks.ModelDoc2
    Dim swEquationMgr  As SldWorks.EquationMgr
    Dim longEquation   As Long

    Set SwApp = Application.SldWorks
    Set Part = SwApp.ActiveDoc

    Set swEquationMgr = Part.GetEquationMgr
    If swEquationMgr Is Nothing Then ErrorMsg SwApp, "Failed to get the equation manager"

    'Add a global variable assignment at index, 0, to all configurations
    longEquation = swEquationMgr.Add3(0, ""A"" = 2in", True, swAllConfiguration, Empty)
    If longEquation <> 0 Then ErrorMsg SwApp, "Failed to add a global variable assignment"

    'Add a dimension equation at index, 1, to all configurations
    longEquation = swEquationMgr.Add3(1, ""D1@Boss-Extrude1"" = 0.05in", True, swAllConfiguration, Empty)
    If longEquation <> 1 Then ErrorMsg SwApp, "Failed to add a dimension equation"

    'Modify dimension equation at index, 1, in all configurations
    longEquation = swEquationMgr.SetEquationAndConfigurationOption(1, ""D1@Boss-Extrude1"" = 0.07in", swAllConfiguration, Empty)
    If longEquation <> 1 Then ErrorMsg SwApp, "Failed to modify a dimension equation"

End Sub

Function ErrorMsg(SwApp As Object, Message As String)
    SwApp.SendMsgToUser2 Message, 0, 0
    SwApp.RecordLine ""**** WARNING - General"
    SwApp.RecordLine ""**** " & Message
    SwApp.RecordLine ""
End Function

```

Add and Modify Equations Example (VB.NET)

This example shows how to add equations to multiple configurations of a part created in SOLIDWORKS 2014 or later.

Preconditions: Open a part created in SOLIDWORKS 2014 or later that has a **Boss-Extrude1** feature and multiple configurations.

Postconditions:

1. Adds two equations to all configurations.
2. Modifies the equation at index 1.
3. Click **Tools > Equations** and examine each configuration in the dialog.

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Sub main()
```

```
        Dim Part As ModelDoc2
        Dim swEquationMgr As EquationMgr
        Dim longEquation As Long
```

```
        Part = SwApp.ActiveDoc
```

```
        swEquationMgr = Part.GetEquationMgr
        If swEquationMgr Is Nothing Then ErrorMsg(SwApp, "Failed to get the equation manager")
```

```
        'Add a global variable assignment at index, 0, to all configurations
        longEquation = swEquationMgr.Add3(0, ""A"" = 2in", True, swInConfigurationOpts_e.swAllConfiguration, Nothing)
        If longEquation <> 0 Then ErrorMsg(SwApp, "Failed to add a global variable assignment")
```

```
        'Add a dimension equation at index, 1, to all configurations
        longEquation = swEquationMgr.Add3(1, ""D1@Boss-Extrude1"" = 0.05in", True, swInConfigurationOpts_e.swAllConfiguration,
Nothing)
        If longEquation <> 1 Then ErrorMsg(SwApp, "Failed to add a dimension equation")
```

```
        'Modify dimension equation at index, 1, in all configurations
        longEquation = swEquationMgr.SetEquationAndConfigurationOption(1, ""D1@Boss-Extrude1"" = 0.07in",
swInConfigurationOpts_e.swAllConfiguration, Nothing)
        If longEquation <> 1 Then ErrorMsg(SwApp, "Failed to modify a dimension equation")
```

```
    End Sub
```

```
    Sub ErrorMsg(ByVal SwApp As Object, ByVal Message As String)
        SwApp.SendMsgToUser2(Message, 0, 0)
        SwApp.RecordLine("**** WARNING - General")
        SwApp.RecordLine("**** " & Message)
        SwApp.RecordLine("")
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Add Regular Explode Step Example (C#)

This example shows how to add a regular (translate and rotate) explode step to an explode view.

```
//-----  
// Preconditions:  
// 1. Open public_documents\samples\tutorial\api\RegularExplodeStep.sldasm.  
// 2. Open an Immediate window.  
//  
// Postconditions:  
// 1. Creates an explode view.  
// 2. Deletes the first explode step.  
// 3. Adds a regular explode step.  
// 4. Inspect the Immediate window and the graphics area.  
//  
// Note: Because the model is used elsewhere, do not save any changes.  
//-----
```

Add Regular Explode Step Example (VBA)

This example shows how to add a regular (translate and rotate) explode step to an explode view.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\RegularExplodeStep.sldasm.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Creates an explode view.
' 2. Deletes the first explode step.
' 3. Adds a regular explode step.
' 4. Inspect the Immediate window and the graphics area.
'
' Note: Because the model is used elsewhere, do not save any changes.
'-----

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.AssemblyDoc
Dim config As SldWorks.Configuration
Dim swMdl As SldWorks.ModelDoc2
Dim explStep As SldWorks.ExplodeStep
Dim num As Double
Dim comp As SldWorks.Component2
Dim var As Variant
Dim transDir As SldWorks.Edge
Dim angleDir As SldWorks.Feature
Dim obj As SldWorks.Component2
Dim steps As Variant
Dim nestedStep As SldWorks.ExplodeStep
Dim boolstatus As Boolean
Dim i As Long, j As Long
Dim errCode As Long

```

Sub main()

```

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set config = swModel.ConfigurationManager.ActiveConfiguration
    Set swMdl = swModel

    Call swModel.AutoExplode

    Set explStep = config.GetExplodeStep(0)

    'If the first step is a subassembly explode step, get the nested explode steps
    If explStep.ExplodeStepType = swAssemblyExplodeStepType_SubAssembly Then
        Debug.Print " Subassembly explode step" & explStep.Name
        steps = explStep.GetSubAssemblyExplodeSteps()

        For j = 0 To UBound(steps)
            Set nestedStep = steps(j)
            Debug.Print "    Name: " & nestedStep.Name
            Debug.Print "    Explode distance: " & nestedStep.ExplodeDistance
            Debug.Print "    Explode rotation angle: " & nestedStep.RotationAngle
        Next

    End If

    config.DeleteExplodeStep (explStep.Name)

    'Select the components to move
    boolstatus = swModel.Extension.SelectByID("Testimo1-2@RegularExplodeStep", "COMPONENT", 0, 0, 0, True, 1, Nothing, 0)
    boolstatus = swModel.Extension.SelectByID("Testimo1-3@RegularExplodeStep", "COMPONENT", 0, 0, 0, True, 1, Nothing, 0)
    boolstatus = swModel.Extension.SelectByID("Testimo1-4@RegularExplodeStep", "COMPONENT", 0, 0, 0, True, 1, Nothing, 0)

    'Select the axis of explode direction
    boolstatus = swModel.Extension.SelectByRay(9.95048714770577E-02, 0.104317306113359, 5.68280010933222E-02, -2.53410890435057E-03, -0.389900775392565, 0.920853388786911, 6.25890574839405E-03, 1, True, 2, 0)

    num = 3.1415 / 3
    Set explStep = config.AddExplodeStep2(0.5, -1, True, num, -1, True, False, True, errCode)

    Call swMdl.EditRebuild3

    var = explStep.GetComponents()

    Debug.Print "Explode step: " & explStep.Name
    Debug.Print "Explode distance (m): " & explStep.ExplodeDistance
    Debug.Print "Explode step type as defined in swAssemblyExplodeStepType_e: " & explStep.ExplodeStepType

```

```
Debug.Print "Rotation angle (rad): " & explStep.RotationAngle
Debug.Print "Reverse rotation direction? " & explStep.ReverseRotationDirection
Debug.Print "Reverse translation direction? " & explStep.ReverseTranslationDirection
Debug.Print "Rotate about each component's origin? " & explStep.RotateAboutEachComponentOrigin
Debug.Print "Automatically space components on drag? " & explStep.AutoSpaceComponentsOnDrag

    Debug.Print "Components to move:"
    For i = 0 To UBound(var)
        Set comp = var(i)
        Debug.Print " " & comp.Name2
    Next
```

End Sub

Add Regular Explode Step Example (VB.NET)

This example shows how to add a regular (translate and rotate) explode step to an explode view.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\RegularExplodeStep.sldasm.  
' 2. Open an Immediate window.  
'  
' Postconditions:  
' 1. Creates an explode view.  
' 2. Deletes the first explode step.  
' 3. Adds a regular explode step.  
' 4. Inspect the Immediate window and the graphics area.  
'  
' Note: Because the model is used elsewhere, do not save any changes.  
'-----
```

Add Highlighting to or Remove Highlight from Edges Example (VBA)

This example shows how to highlight or remove highlight from the selected face's edges.

```
'-----  
' Preconditions:  
' 1. Open a part.  
' 2. Select a face.  
'  
' Postconditions:  
' 1. Displays a message box asking whether to highlight  
'    the edges on the selected face.  
' 2. Click Yes.  
' 3. Examine the part.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swFace As SldWorks.Face  
    Dim vEdgeArr As Variant  
    Dim vEdge As Variant  
    Dim swEdge As SldWorks.Edge  
    Dim nResponse As Integer  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swFace = swSelMgr.GetSelectedObject6(1, -1)  
  
    nResponse = MsgBox("Highlight edges on selected face?", vbYesNo)  
  
    swModel.ClearSelection2 True  
  
    vEdgeArr = swFace.GetEdges  
    For Each vEdge In vEdgeArr  
        Set swEdge = vEdge  
        If nResponse = vbYes Then  
            swEdge.Display 2, 1#, 0#, 0#, True  
        Else  
            swEdge.Display 0, 0#, 0#, 0#, False  
        End If  
    Next vEdge  
  
End Sub
```

Add Menu and Menu Item Example (C#)

This example shows how to add a menu and menu item to a part.

```
//-----
// Preconditions:
// 1. Verify that the SOLIDWORKS API SDK add-in templates for your version
// of Microsoft Visual Studio are installed.
// 2. In Microsoft Visual Studio, create a C# project using the
// SwSharpAddin template.
// 3. Name the project AddMenuItemCSharp.
// 4. Copy and paste SwAddin into SwAddin.cs of your project.
// 5. Click Project > AddMenuItemCSharp Properties > Debug.
// 6. Select Start external program and type the pathname to your
// SOLIDWORKS executable.
// 7. Replace the images array elements with the pathnames to
// your image files.
// 8. Press F5 to start debugging this add-in.
//
// Postconditions:
// 1. In SOLIDWORKS, click File > New > Part > OK.
// 2. Click Tools > MyMenu. Note the size of
// the MyMenuItem button.
// 3. Click MyMenuItem and click OK to close the message box.
// 4. Exit SOLIDWORKS.
// 5. Change the size of the text and other items on your computer.
// For example, in Windows 7:
// a. Click Start > Control Panel > Appearance and
// Personalization > Display.
// b. Select a different size.
// c. Click Apply.
// d. Click Log off now.
// e. Log back in.
// 6. Start SOLIDWORKS.
// 7. Repeat steps 1 and 2.
//-----
//SwAddin
using System;
using System.Runtime.InteropServices;
using System.Collections;
using System.Reflection;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swpublished;
using SolidWorks.Interop.swconst;
using SolidWorksTools;
using SolidWorksTools.File;
using System.Collections.Generic;
using System.Diagnostics;
using System.Windows.Forms;

namespace AddMenuItemCSharp
{
    /// <summary>
    /// Summary description for AddMenuItemCSharp.
    /// </summary>
    [Guid("ff413b4-df9a-4d58-b4c2-171c74e72971"), ComVisible(true)]
    [SwAddin(
        Description = "AddMenuItemCSharp description",
        Title = "AddMenuItemCSharp",
        LoadAtStartup = true
    )]
    public class SwAddin : ISwAddin
    {
        #region Local Variables
        ISldWorks iSwApp = null;
        int addinID = 0;

        #region Event Handler Variables
        Hashtable openDocs = new Hashtable();
        SolidWorks.Interop.sldworks.SldWorks SwEventPtr = null;
        #endregion

        // Public Properties
        public ISldWorks SwApp
        {
            get { return iSwApp; }
        }

        public Hashtable OpenDocs
        {
            get { return openDocs; }
        }

        #endregion

        #region SolidWorks Registration
        [ComRegisterFunctionAttribute]
        public static void RegisterFunction(Type t)
        {
            #region Get Custom Attribute: SwAddinAttribute
            SwAddinAttribute SwAttr = null;
            Type type = typeof(SwAddin);

            foreach (System.Attribute attr in type.GetCustomAttributes(false))
            {
                if (attr is SwAddinAttribute)
                {
                    SwAttr = attr as SwAddinAttribute;
                    break;
                }
            }

            #endregion

            try
            {
                Microsoft.Win32.RegistryKey hklm = Microsoft.Win32.Registry.LocalMachine;
                Microsoft.Win32.RegistryKey hkcu = Microsoft.Win32.Registry.CurrentUser;
            }
        }
    }
}
```

```

        string keyname = "SOFTWARE\\SolidWorks\\Addins\\" + t.GUID.ToString() + "}";
        Microsoft.Win32.RegistryKey addinkey = hkln.CreateSubKey(keyname);
        addinkey.SetValue(null, 0);

        addinkey.SetValue("Description", Swattr.Description);
        addinkey.SetValue("Title", Swattr.Title);

        keyname = "Software\\SolidWorks\\AddInsStartup\\" + t.GUID.ToString() + "}";
        addinkey = hkcu.CreateSubKey(keyname);
        addinkey.SetValue(null, Convert.ToInt32(Swattr.LoadAtStartup), Microsoft.Win32.RegistryValueKind.DWord);
    }
    catch (System.NullReferenceException nl)
    {
        Console.WriteLine("There was a problem registering this dll: Swattr is null. \n\" + nl.Message + "\");
        System.Windows.Forms.MessageBox.Show("There was a problem registering this dll: Swattr is null. \n\" + nl.Message + "\");
    }

    catch (System.Exception e)
    {
        Console.WriteLine(e.Message);

        System.Windows.Forms.MessageBox.Show("There was a problem registering the function: \n\" + e.Message + "\");
    }
}

[ComUnregisterFunctionAttribute]
public static void UnregisterFunction(Type t)
{
    try
    {
        Microsoft.Win32.RegistryKey hkln = Microsoft.Win32.Registry.LocalMachine;
        Microsoft.Win32.RegistryKey hkcu = Microsoft.Win32.Registry.CurrentUser;

        string keyname = "SOFTWARE\\SolidWorks\\Addins\\" + t.GUID.ToString() + "}";
        hkln.DeleteSubKey(keyname);

        keyname = "Software\\SolidWorks\\AddInsStartup\\" + t.GUID.ToString() + "}";
        hkcu.DeleteSubKey(keyname);
    }
    catch (System.NullReferenceException nl)
    {
        Console.WriteLine("There was a problem unregistering this dll: " + nl.Message);
        System.Windows.Forms.MessageBox.Show("There was a problem unregistering this dll: \n\" + nl.Message + "\");
    }
    catch (System.Exception e)
    {
        Console.WriteLine("There was a problem unregistering this dll: " + e.Message);
        System.Windows.Forms.MessageBox.Show("There was a problem unregistering this dll: \n\" + e.Message + "\");
    }
}

#endregion

#region ISwAddin Implementation
public SwAddin()
{
}

public bool ConnectToSW(object ThisSW, int cookie)
{
    iSwApp = (ISldWorks)ThisSW;
    addinID = cookie;

    //Set up callbacks
    iSwApp.SetAddinCallbackInfo2(0, this, addinID);

    #region Setup the menu item
    AddMenuItem();
    #endregion

    #region Set up the Event Handlers
    SwEventPtr = (SolidWorks.Interop.sldworks.SldWorks)iSwApp;
    openDocs = new Hashtable();
    AttachEventHandlers();
    #endregion

    return true;
}

public bool DisconnectFromSW()
{
    DetachEventHandlers();

    System.Runtime.InteropServices.Marshal.ReleaseComObject(iSwApp);
    iSwApp = null;
    // The add-in must call GC.Collect() here to retrieve all managed code pointers
    GC.Collect();
    GC.WaitForPendingFinalizers();

    GC.Collect();
    GC.WaitForPendingFinalizers();

    return true;
}
#endregion

#region UI Methods
public void AddMenuItem()
{
    Assembly thisAssembly = default(Assembly);
    int menuId = 0;
    string[] images = new string[3];

    thisAssembly = System.Reflection.Assembly.GetAssembly(this.GetType());

    images[0] = "K:\\api\\HELP\\Sample Macros\\Sw2016\\b2\\ISldWorksAddMenuItem5\\AddMenuItemCSharp\\AddMenuItemCSharp\\Small.bmp";
    images[1] = "K:\\api\\HELP\\Sample Macros\\Sw2016\\b2\\ISldWorksAddMenuItem5\\AddMenuItemCSharp\\AddMenuItemCSharp\\Medium.bmp";
    images[2] = "K:\\api\\HELP\\Sample Macros\\Sw2016\\b2\\ISldWorksAddMenuItem5\\AddMenuItemCSharp\\AddMenuItemCSharp\\Large.bmp";

    menuId = SwApp.AddMenu((int)swDocumentTypes_e.swDocPART, "MyMenu", 0);
    menuId = iSwApp.AddMenuItem5((int)swDocumentTypes_e.swDocPART, addinID, "MyMenuItem@MyMenu", 0, "MyMenuCallback", "MyMenuEnableMethod", "My menu item", images);

    thisAssembly = null;
}
#endregion

#region UI Callbacks
public void MyMenuCallback()

```

```

{
    MessageBox.Show("Callback function called.");
}
#endregion

#region Event Methods
public bool AttachEventHandlers()
{
    AttachSwEvents();
    // Listen for events on all currently open docs
    AttachEventsToAllDocuments();
    return true;
}

private bool AttachSwEvents()
{
    try
    {
        SwEventPtr.ActiveDocChangeNotify += new DSldWorksEvents_ActiveDocChangeNotifyEventHandler(ActiveDocChangeNotify);
        SwEventPtr.DocumentLoadNotify2 += new DSldWorksEvents_DocumentLoadNotify2EventHandler(DocumentLoadNotify2);
        SwEventPtr.FileNewNotify2 += new DSldWorksEvents_FileNewNotify2EventHandler(FileNewNotify2);
        SwEventPtr.ActiveModelDocChangeNotify += new DSldWorksEvents_ActiveModelDocChangeNotifyEventHandler(ActiveModelDocChangeNotify);
        SwEventPtr.FileOpenPostNotify += new DSldWorksEvents_FileOpenPostNotifyEventHandler(FileOpenPostNotify);
        return true;
    }
    catch (Exception e)
    {
        Console.WriteLine(e.Message);
        return false;
    }
}

private bool DetachSwEvents()
{
    try
    {
        SwEventPtr.ActiveDocChangeNotify -= new DSldWorksEvents_ActiveDocChangeNotifyEventHandler(ActiveDocChangeNotify);
        SwEventPtr.DocumentLoadNotify2 -= new DSldWorksEvents_DocumentLoadNotify2EventHandler(DocumentLoadNotify2);
        SwEventPtr.FileNewNotify2 -= new DSldWorksEvents_FileNewNotify2EventHandler(FileNewNotify2);
        SwEventPtr.ActiveModelDocChangeNotify -= new DSldWorksEvents_ActiveModelDocChangeNotifyEventHandler(ActiveModelDocChangeNotify);
        SwEventPtr.FileOpenPostNotify -= new DSldWorksEvents_FileOpenPostNotifyEventHandler(FileOpenPostNotify);
        return true;
    }
    catch (Exception e)
    {
        Console.WriteLine(e.Message);
        return false;
    }
}

}

public void AttachEventsToAllDocuments()
{
    ModelDoc2 modDoc = (ModelDoc2)iSwApp.GetFirstDocument();
    while (modDoc != null)
    {
        if (!openDocs.Contains(modDoc))
        {
            AttachModelDocEventHandler(modDoc);
        }
        else if (openDocs.Contains(modDoc))
        {
            bool connected = false;
            DocumentEventHandler docHandler = (DocumentEventHandler)openDocs[modDoc];
            if (docHandler != null)
            {
                connected = docHandler.ConnectModelViews();
            }
        }
        modDoc = (ModelDoc2)modDoc.GetNext();
    }
}

public bool AttachModelDocEventHandler(ModelDoc2 modDoc)
{
    if (modDoc == null)
        return false;

    DocumentEventHandler docHandler = null;

    if (!openDocs.Contains(modDoc))
    {
        switch (modDoc.GetType())
        {
            case (int)swDocumentTypes_e.swDocPART:
            {
                docHandler = new PartEventHandler(modDoc, this);
                break;
            }
            case (int)swDocumentTypes_e.swDocASSEMBLY:
            {
                docHandler = new AssemblyEventHandler(modDoc, this);
                break;
            }
            case (int)swDocumentTypes_e.swDocDRAWING:
            {
                docHandler = new DrawingEventHandler(modDoc, this);
                break;
            }
            default:
            {
                // Unsupported document type
                return false;
            }
        }
    }
    docHandler.AttachEventHandlers();
    openDocs.Add(modDoc, docHandler);
}
return true;
}

}

public bool DetachModelEventHandler(ModelDoc2 modDoc)
{
    DocumentEventHandler docHandler;
    docHandler = (DocumentEventHandler)openDocs[modDoc];
    openDocs.Remove(modDoc);
    modDoc = null;
    docHandler = null;
    return true;
}

```

```

}

public bool DetachEventHandlers()
{
    DetachSwEvents();

    // Close events on all currently open documents
    DocumentEventHandler docHandler;
    int numKeys = openDocs.Count;
    object[] keys = new Object[numKeys];

    // Remove all document event handlers
    openDocs.Keys.CopyTo(keys, 0);
    foreach (ModelDoc2 key in keys)
    {
        docHandler = (DocumentEventHandler)openDocs[key];
        // Remove the pair from the hash
        docHandler.DetachEventHandlers();
        docHandler = null;
    }
    return true;
}
#endregion

#region Event Handlers
// Events
public int ActiveDocChangeNotify()
{
    return 0;
}

public int DocumentLoadNotify2(string docTitle, string docPath)
{
    return 0;
}

public int FileOpenPostNotify(string FileName)
{
    AttachEventsToAllDocuments();
    return 0;
}

public int FileNewNotify2(object newDoc, int docType, string templateName)
{
    AttachEventsToAllDocuments();
    return 0;
}

public int ActiveModelDocChangeNotify()
{
    return 0;
}
#endregion
}
}

```